STELLAR OBJECT CLASSIFICATION

Roopesh J

roopeshjayaprakash@gmail.com

ABSTRACT

This project aims at developing comprehensive machine learning models to accurately classify celestial objects as stars, galaxies, or quasars. By utilizing these models, we aimed to improve astronomical cataloging, research directions, and our understanding of the universe.

INTRODUCTION

The classification of celestial objects—such as stars, galaxies, and quasars—is a fundamental task in Astrophysics and Observational astronomy. Given the vast amount of Astronomical data collected through large-scale sky surveys, automated classification using machine learning has become increasingly important. This project leverages machine learning techniques to classify celestial objects based on key astronomical features extracted from telescopic observations.

THEORETICAL BACKGROUND AND METHODOLOGY

Astronomical Coordinate System

The position of celestial objects in the sky is specified using the **Right Ascension (RA)** and **Declination (Dec)** coordinate system: 1. Right Ascension **(α)**: Similar to longitude on Earth, RA measures an object's angular distance eastward along the celestial equator. 2. Declination **(δ)**: Analogous to latitude, Dec measures the angular distance north or south of the celestial equator. These coordinates are defined for the J2000 epoch, a standard reference frame used in astronomy to account for the slow precession of Earth's rotation axis over time.

Photometric Features

Observing celestial objects involves capturing their electromagnetic spectrum at different wavelengths. The **Sloan Digital Sky Survey (SDSS)** photometric system uses five broad-band filters i.e. **ultraviolet (u), green (g), red (r), near-infrared (i), and infrared (z)**—to measure the brightness of objects at specific wavelengths. These filters help distinguish between different astronomical bodies based on their color indices and spectral energy distributions.

Spectroscopic Features

Spectroscopic data provide additional insights into the physical properties of celestial objects. The **redshift** value, for instance, is a critical feature in classifying galaxies and quasars. **Redshift (z)**: Measures the increase in the wavelength of light due to the expansion of the universe (Doppler effect). Higher redshift values typically indicate objects that are farther away and moving away from us at greater speeds. **Spectroscopic Object ID (spec_obj_ID)**: Unique identifiers help ensure that repeated observations of the same object are correctly classified.

SDSS Data Collection and Identification System

The SDSS uses a systematic data collection process, assigning unique identifiers to each object and observation: **Object ID (obj_ID)**: A unique identifier for each astronomical object in the catalog. **Run ID** and **Rerun ID**: Track the specific imaging scan and any reprocessing applied to the data. **Field ID** and **Camera Column (cam_col)**: Indicate the region of the sky covered in a given scan. **Plate ID, MJD, and Fiber ID**: Essential metadata for spectroscopic observations, identifying the observation date, telescope plate, and fiber used for data collection. SDSS collects data with the help of UGRIZ filters.

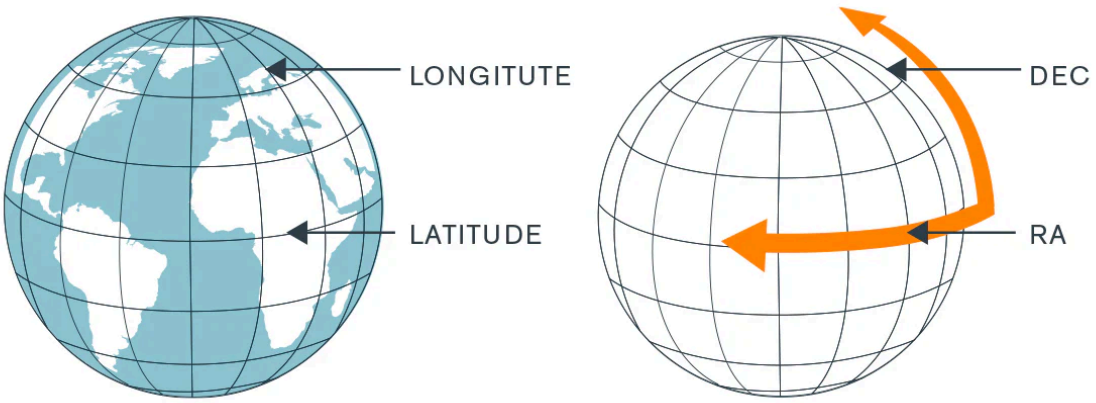


Image 1: Right Ascension and Declination

Classification of Celestial Objects

The primary objective is to classify objects into three categories: **Stars**: Luminous celestial bodies undergoing nuclear fusion. Typically exhibit minimal redshift and have distinct brightness patterns across photometric filters.**Galaxies**: Massive systems of stars, gas, and dark matter bound by gravity. Exhibit a range of redshifts based on their distance from Earth. **Quasars**: Extremely luminous active galactic nuclei powered by supermassive black holes. They exhibit high redshifts and strong emission lines in their spectra. By leveraging a combination of photometric, spectroscopic, and positional features, machine learning models can efficiently classify celestial objects and support large-scale astronomical research.

Objectives:

- To develop machine learning models to classify celestial objects into Stars, Galaxies, and Quasars.
- To facilitate efficient automated classification, insights into distinguishing features, and optimized data processing for large-scale surveys through the predictive models, data visualizations
- To compare analysis of various algorithms.

Methodology:

Tools used for the project:

1. IDE: VS Code
2. Python Notebook
3. Libraries used: Numpy, Pandas, Sklearn
4. User Interface using Streamlit
5. Dashboard using PowerBI
6. Models used for Classification: Decision Tree, Gaussian Naive Bayes, K Nearest Neighbours, Support Vector Machine, Random Forest, XGBoost, ADA Boost, Gradient Boost

Research Methodology:

1. Data Collection: Used Secondary Data Collected by Centre for Astrophysics and Supercomputing (CAS), Swinburne, Australia
2. Preliminary Data Analysis: Checking for basic patterns in the data, errors in entry, schema, data types
3. Exploratory Data Analysis: checking for anomalies, distribution, multicollinearity and other important features which will be important to take note during feature engineering.

Feature Description

1. obj_ID: Object Identifier, the unique value that identifies the object in the image catalogue used by CAS
2. alpha: Right Ascension angle (at J2000 epochs)
3. delta: Declination angle (at J2000 epoch)
4. u: Ultraviolet filter in the photometric system
5. g: Green filter in the photometric system
6. r: Red filter in the photometric system
7. i: Near Infrared filter in the photometric system
8. z: Infrared filter in the photometric system
9. run_ID: Run Number used to identify the specific scan
10. rereun_ID: Rerun Number to specify how the image was processed
11. cam_col: Camera column to identify the scanline within the run
12. field_ID: Field number to identify each field
13. spec_obj_ID: Unique ID used for optical spectroscopic objects (this means that 2 different observations with the same spec_obj_ID must share the output class)
14. class: object class (galaxy, star or quasar object)
15. redshift: redshift value based on the increase in wavelength
16. plate ID: plate ID, identifies each plate in SDSS
17. MJD: Modified Julian Date, used to indicate when a given piece of SDSS data was taken
18. fiber_ID: fiber ID that identifies the fiber that pointed the light at the focal plane in each observation

IMPLEMENTATION AND ANALYSIS

Preliminary Analysis

Included checking for the data types of the features, checking for null values, duplicates, distribution and presence of outliers.

Checking for Data Types, Null Values, Dimension of the Dataset

```
df.info()
Output>>
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 12 columns):
 #   Column    Non-Null Count   Dtype
---  ------    --------------   -----
 0   alpha     100000 non-null  float64
 1   delta     100000 non-null  float64
 2   u         100000 non-null  float64
 3   g         100000 non-null  float64
 4   r         100000 non-null  float64
 5   i         100000 non-null  float64
 6   z         100000 non-null  float64
 7   cam_col   100000 non-null  int64
 8   class     100000 non-null  object
 9   redshift  100000 non-null  float64
 10  plate     100000 non-null  int64
 11  MJD       100000 non-null  int64
dtypes: float64(8), int64(3), object(1)
memory usage: 9.2+ MB
```

There are 100000 entries and 12 columns in the Dataset; no null values detected.

Checking for Duplicates in the dataset

```
df.duplicated().sum()
Output>>np.int64(0)
```

No Duplicates detected

Checking for Distribution of the features

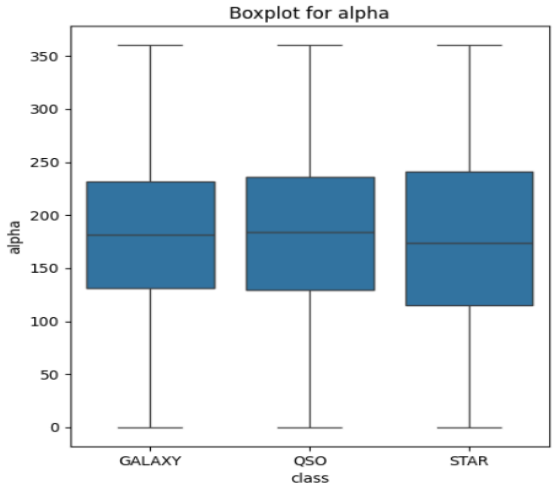
```
from scipy.stats import shapiro
for i in df_num:
   stat, p_value = shapiro(df[i])
   print(f'Results for {i}')
   print(f"Shapiro-Wilk Test Statistic:{stat}")
   print(f"p-value: {p_value}")
   if p_value > 0.05:
       print("The data appears to follow a normal distribution.")
   else:
       print("The data does not follow a normal distribution.")
   print('-'*50)
Output>>Results for alpha
Shapiro-Wilk Test Statistic:0.9563737789475277
p-value: 1.022134013278 66e-90
The data does not follow a normal distribution.
--------------------------------------------------
Results for delta
Shapiro-Wilk Test Statistic:0.9621126997592861
p-value: 2.789043649450643e-87
The data does not follow a normal distribution.
--------------------------------------------------
Results for u
Shapiro-Wilk Test Statistic:0.0072420271708125705
p-value: 3.2684703749834186e-184
The data does not follow a normal distribution.
--------------------------------------------------
Results for g
Shapiro-Wilk Test Statistic:0.0060387392646260585
p-value: 2.9624645513191885e-184
The data does not follow a normal distribution.
--------------------------------------------------
Results for r
Shapiro-Wilk Test Statistic:0.9624677187770521
p-value: 4.703216475993495e-87
The data does not follow a normal distribution.
--------------------------------------------------
Results for i
Shapiro-Wilk Test Statistic:0.9769729679395429
p-value: 1.0833572379717157e-75
The data does not follow a normal distribution.
--------------------------------------------------
Results for z
Shapiro-Wilk Test Statistic:0.004880623122459404
p-value: 2.695308532270801e-184
The data does not follow a normal distribution.
Results for cam_col
```

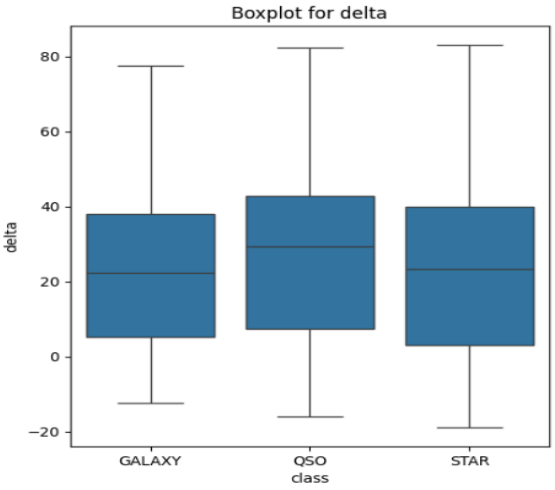
```
Shapiro-Wilk Test Statistic:0.922842719959489
p-value: 2.686081812971434e-105
The data does not follow a normal distribution.
Results for redshift
Shapiro-Wilk Test Statistic:0.7402545511594765
p-value: 5.051986435440 55e-140
The data does not follow a normal distribution.
Results for plate
Shapiro-Wilk Test Statistic:0.9685683947435505
p-value: 7.797917551405964e-83
The data does not follow a normal distribution.
Results for MJD
Shapiro-Wilk Test Statistic:0.9548883552781265
p-value: 1.52724062348084 66e-91
The data does not follow a normal distribution.
```

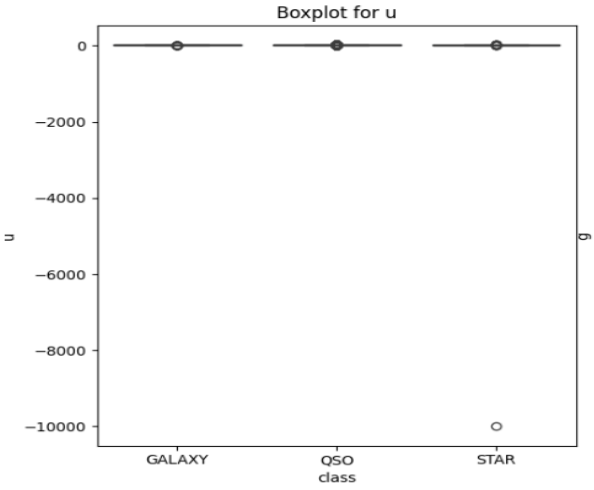
From Using Shapiro Analysis we got to know that all of the features do not follow normal distribution and are highly skewed. Though it is not necessary for features to follow Gaussian Distribution as it is a Multinomial Classification problem and unlike Linear and Logistic regression (and Gaussian bayes) it does not assume normality in the features.

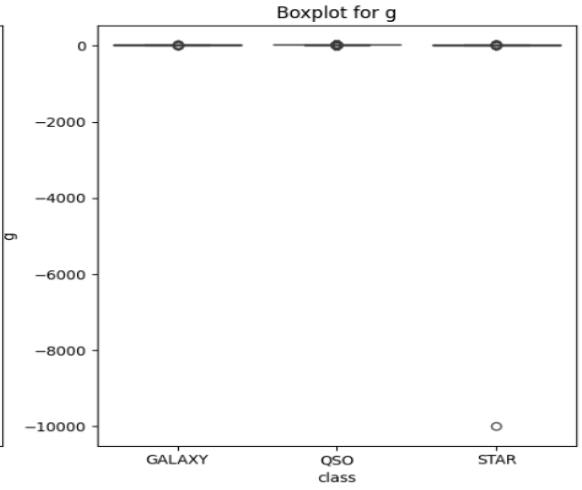
Checking for Outliers of numerical features using Boxplot:

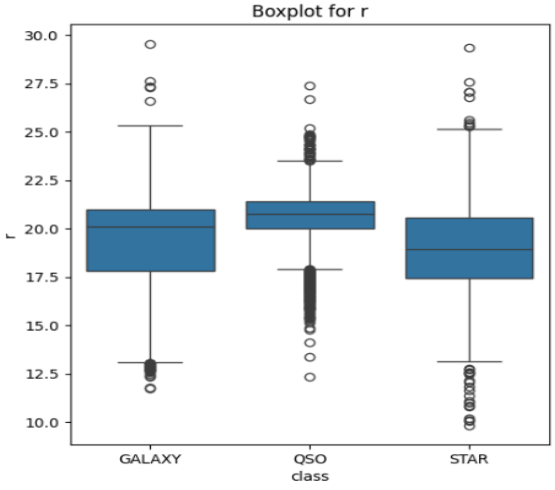
```
#Checking for outliers using boxplot
plt.figure(figsize=(12, 35))#Defining size of the plots
plt.subplot(5,2,1)
plt.title('Boxplot for alpha')#Giving title to each plots
sns.boxplot(x=df['class'], y=df['alpha'])
plt.subplot(5,2,2)
plt.title('Boxplot for delta')
sns.boxplot(x=df['class'], y=df['delta'])
plt.subplot(5,2,3)
plt.title('Boxplot for u')
sns.boxplot(x=df['class'], y=df['u'])
plt.subplot(5,2,4)
plt.title('Boxplot for g')
sns.boxplot(x=df['class'], y=df['g'])
plt.subplot(5,2,5)
plt.title('Boxplot for r')
sns.boxplot(x=df['class'], y=df['r'])
plt.subplot(5,2,6)
plt.title('Boxplot for i')
sns.boxplot(x=df['class'], y=df['i'])
plt.subplot(5,2,7)
plt.title('Boxplot for z')
sns.boxplot(x=df['class'], y=df['z'])
plt.subplot(5,2,8)
plt.title('Boxplot for redshift')
sns.boxplot(x=df['class'], y=df['redshift'])
```

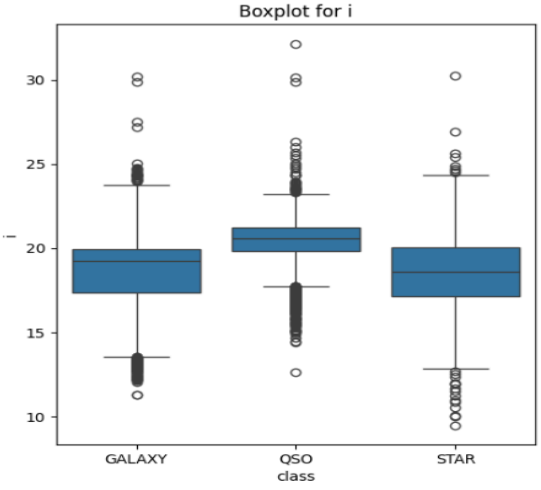

Boxplot for alpha
alpha
350
300
250
200
150
100
50
0
GALAXY
QSO
STAR
class

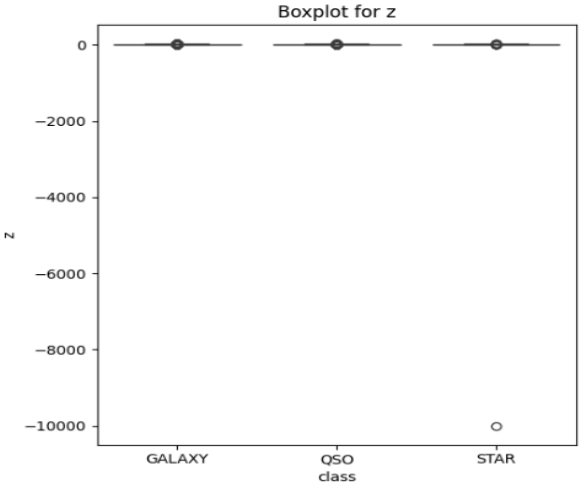

Boxplot for delta
delta
80
60
40
20
0
−20
GALAXY
QSO
STAR
class

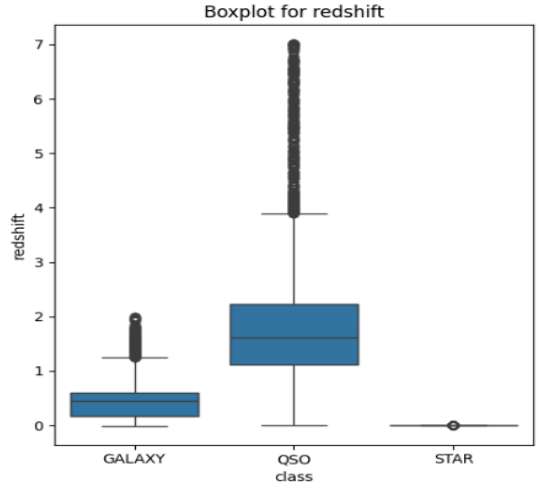

Boxplot for u
u
0
−2000
−4000
−6000
−8000
−10000
GALAXY
QSO
STAR
class


Boxplot for g
g
0
−2000
−4000
−6000
−8000
−10000
GALAXY
QSO
STAR
class


Boxplot for r
r
30.0
27.5
25.0
22.5
20.0
17.5
15.0
12.5
10.0
GALAXY
QSO
STAR
class


Boxplot for i
i
30
25
20
15
10
GALAXY
QSO
STAR
class


Boxplot for z
z
0
−2000
−4000
−6000
−8000
−10000
GALAXY
QSO
STAR
class


Boxplot for redshift
redshift
7
6
5
4
3
2
1
0
GALAXY
QSO
STAR
class

- While in Alpha and Delta no outliers were detected,
- In the features u, g and z, all of the values are near the median, suggesting less variance and the values are almost symmetric.
- Whereas in r, i and redshift there were several outliers detected.

Descriptive Statistics of the Features:

```
df.describe()
```

	alpha	delta	u	g	r	i	z
count	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000
mean	177.629117	24.135305	21.980468	20.531387	19.645762	19.084854	18.668810
std	96.502241	19.644665	31.769291	31.750292	1.854760	1.757895	31.728152
min	0.005528	-18.785328	-9999.000000	-9999.000000	9.822070	9.469903	-9999.000000
25%	127.518222	5.146771	20.352353	18.965230	18.135828	17.732285	17.460677
50%	180.900700	23.645922	22.179135	21.099835	20.125290	19.405145	19.004595
75%	233.895005	39.901550	23.687440	22.123767	21.044785	20.396495	19.921120
max	359.999810	83.000519	32.781390	31.602240	29.571860	32.141470	29.383740

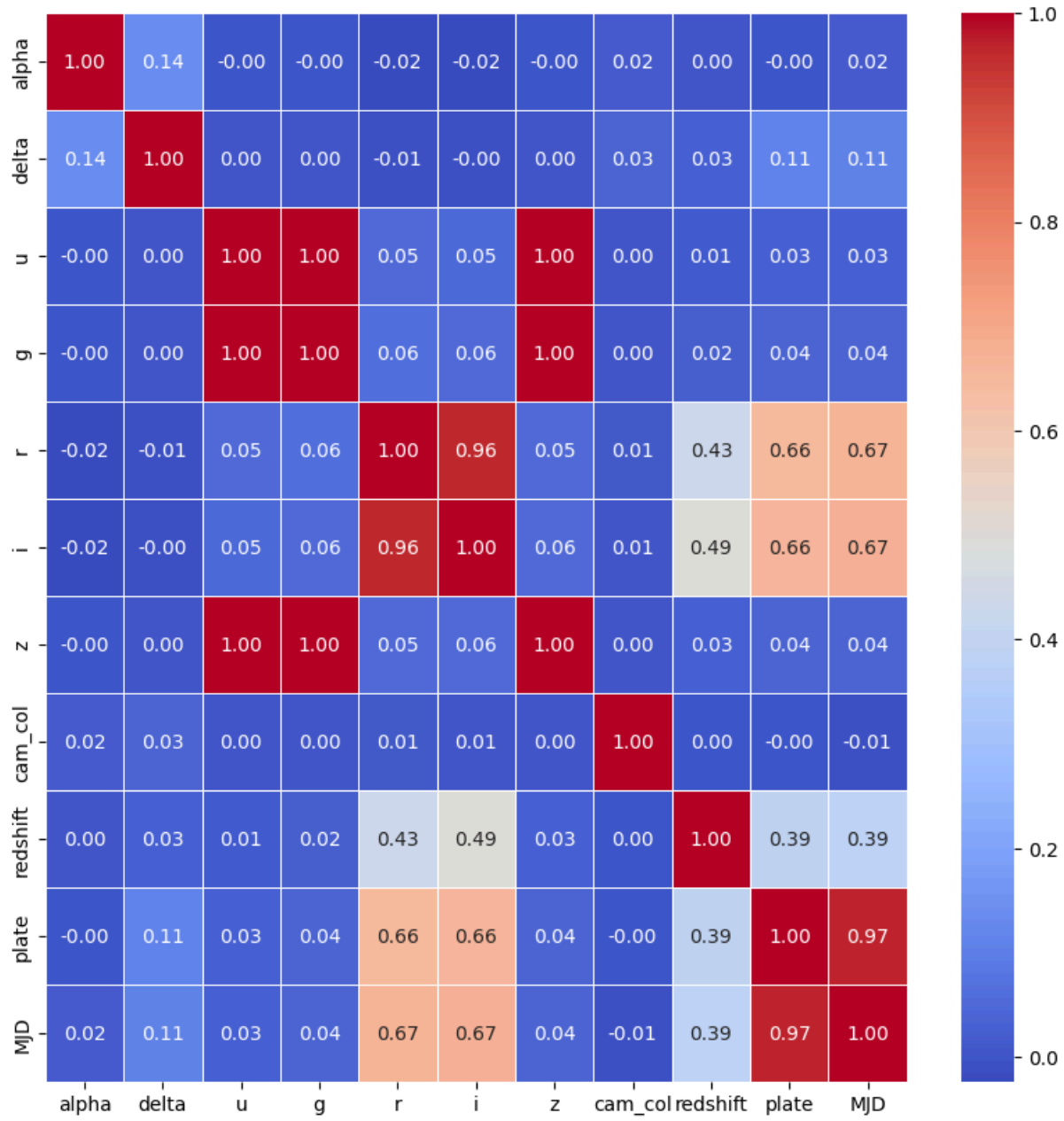
	cam_col	redshift	plate	MJD
count	100000.000000	100000.000000	100000.000000	100000.000000
mean	3.511610	0.576661	5137.009660	55588.647500
std	1.586912	0.730707	2952.303351	1808.484233
min	1.000000	-0.009971	266.000000	51608.000000
25%	2.000000	0.054517	2526.000000	54234.000000
50%	4.000000	0.424173	4987.000000	55868.500000
75%	5.000000	0.704154	7400.250000	56777.000000
max	6.000000	7.011245	12547.000000	58932.000000

Exploratory Data Analysis

Correlation among Features:

Reducing the dimensionality of the Dataset is crucial for improving the performance of the model. Highly correlated features. A common approach for finding it using Heatmaps.

```
plt.figure(figsize=(10, 10))
sns.heatmap(df_num.corr(), annot = True, fmt = ".2f", linewidths = .5,
cmap='coolwarm')
plt.show()
```



In the heat map we can see the boxes in red colour to be highly correlated. By deciding to set a threshold of 0.9 for the correlation, we remove features above the threshold.

```
correlation_matrix = df_num.corr()
threshold = 0.9 # Defined a threshold for high correlation
high_correlation_pairs = [
    (col1, col2, correlation_matrix.loc[col1, col2])
    for col1 in correlation_matrix.columns
    for col2 in correlation_matrix.columns
    if col1 != col2 and abs(correlation_matrix.loc[col1, col2]) > threshold
]
unique_pairs = set(tuple(sorted(pair[:2])) + (pair[2],) for pair in
high_correlation_pairs)
# Display the results
for pair in unique_pairs:
    print(f"Columns: {pair[0]} and {pair[1]} with correlation: {pair[2]:.2f}")
```

```
Output>> Columns: g and z with correlation: 1.00
Columns: g and u with correlation: 1.00
Columns: u and z with correlation: 1.00
Columns: MJD and plate with correlation: 0.97
Columns: i and r with correlation: 0.96
```

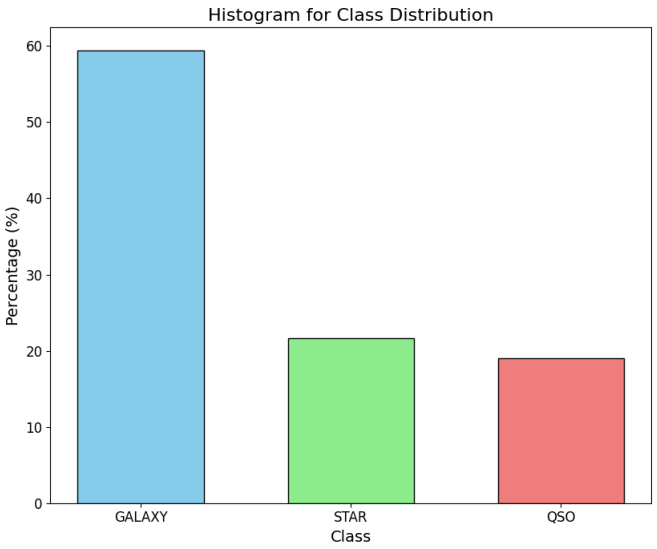
From the above analysis, we found that

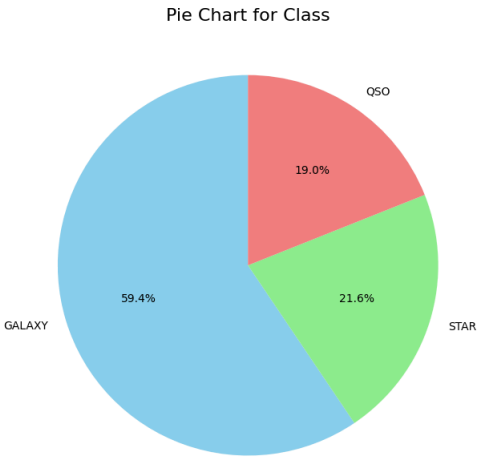
- Perfect Positive correlation exists between features g, u, z
- MJD and Plate, i and r are also highly correlated

Distribution of Target Feature:

In a Classification problem it is important for the feature to follow a Uniform Distribution, so that the model learns all of the classes equally and any kind of bias is avoided. By plotting a Histogram and a pie chart we shall visualise the distribution of the classes.

```
labels=df['class'].value_counts(normalize=True)*100
classlabels = ['GALAXY', 'STAR', 'QSO']
colors = ['skyblue', 'lightgreen', 'lightcoral']
plt.figure(figsize=(16, 7))

# Histogram (left subplot)
plt.subplot(1, 2, 1)  # 1 row, 2 columns, 1st plot
plt.bar(classlabels, labels, color=colors, edgecolor='black', width=0.6)
plt.xlabel('Class', fontsize=14)
plt.ylabel('Percentage (%)', fontsize=14)
plt.title('Histogram for Class Distribution', fontsize=16)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)

# Pie chart (right subplot)
plt.subplot(1, 2, 2)  # 1 row, 2 columns, 2nd plot
plt.pie(labels, labels=classlabels, autopct='%1.1f%%', startangle=90,
colors=colors)
plt.title('Pie Chart for Class', fontsize=16)

# Adjust layout and display
plt.tight_layout()
plt.show()
```





By looking at the plot we get to know that while Stars and Quasars are fairly equally represented, the class Galaxy has the Majority Distribution in the target feature and Uniform Distribution is not present.

Summary of Data Analysis:

By looking into the dataset, we got to know the below following information:

1. Nulls detected - 0
2. Duplicates - 0
3. Data types - Correct
4. Normality - No
5. Irrelevant Features like ID etc. - Yes
6. Outliers - Exists
7. Multicollinearity - Exists
8. Uniformly distributed target class - No

Time Series Analysis:

We have a feature MJD i.e. Modified Julian Date which represents the date of the observation. Using this we can look inside the patterns of QoQ, MoM and YoY observations over Galaxies, Stars and Quasars, to see if the time of the observation has any relation with a particular class and so on.

MJD is nothing but the number from the Julian date in dd/mm/yyyy format converted to the number of days from 17 November 1858.

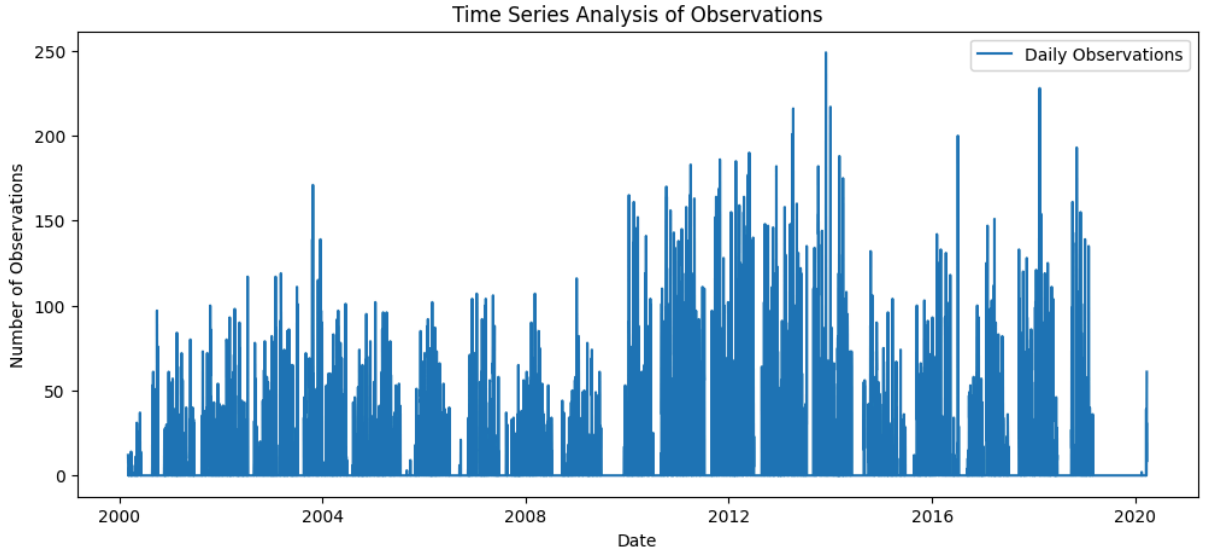
We have to convert MJD to JD (Julian Date) to perform any sort of Time Series Analysis.

```
# Time Series analysis using MJD
# Converting MJd to Date dd-mm-yyyy format
df['Date'] = [t.strftime('%d-%m-%Y') for t in Time(df['MJD'],
format='mjd').to_datetime()]
print(df[['MJD', 'Date']].head())
```

```
    MJD        Date
0  56354  03-03-2013
1  58158  09-02-2018
2  55592  31-01-2011
3  58039  13-10-2017
4  56187  17-09-2012
```

Checking the number of Observations over the years

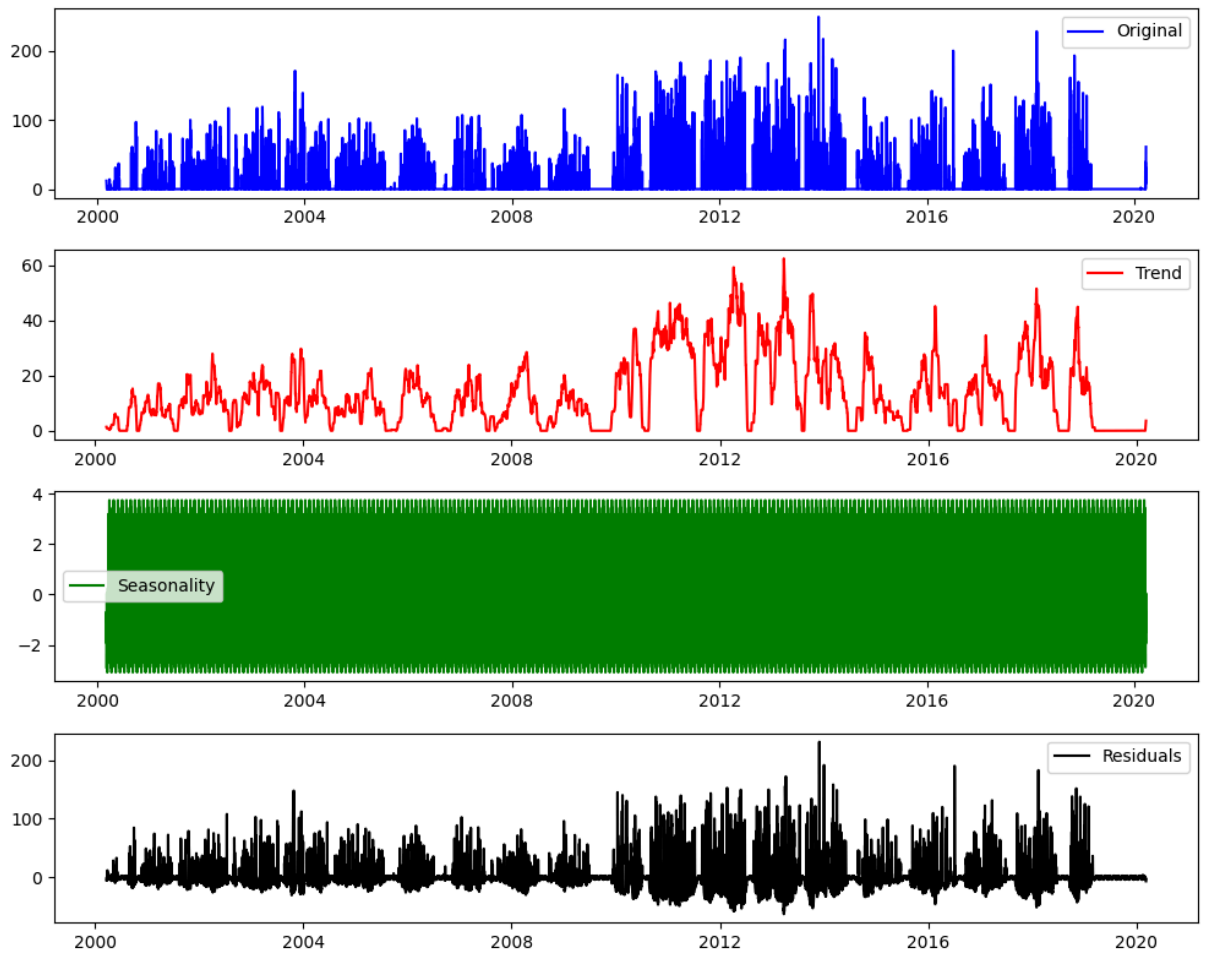
```
df['Date'] = pd.to_datetime(df['Date'], format='%d-%m-%Y')

# Count Observations per Day
daily_counts = df.resample('D', on='Date').count()

# Plot Time Series
plt.figure(figsize=(12, 5))
sns.lineplot(data=daily_counts, x=daily_counts.index, y='class', label='Daily
Observations')
plt.xlabel("Date")
plt.ylabel("Number of Observations")
plt.title("Time Series Analysis of Observations")
plt.legend()
plt.show()
```



Visualising the components of Time Series viz. Trend, Seasonality, Cyclicity and Outliers

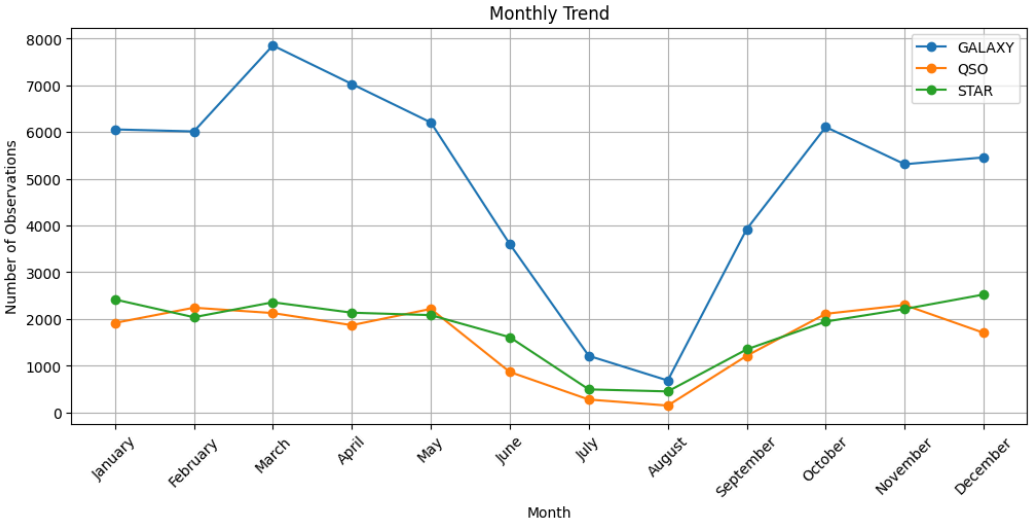
```
# break down of time series into trend, seasonality, and residuals.
daily_counts = daily_counts.rename(columns={'class': 'observations'})
decomposition = seasonal_decompose(daily_counts['observations'],
model='additive', period=30)

plt.figure(figsize=(10, 8))

plt.subplot(411)#Observation
plt.plot(decomposition.observed, label='Original', color='blue')
plt.legend()

plt.subplot(412)#Trend
plt.plot(decomposition.trend, label='Trend', color='red')
plt.legend()

plt.subplot(413)#Seasonality
plt.plot(decomposition.seasonal, label='Seasonality', color='green')
```

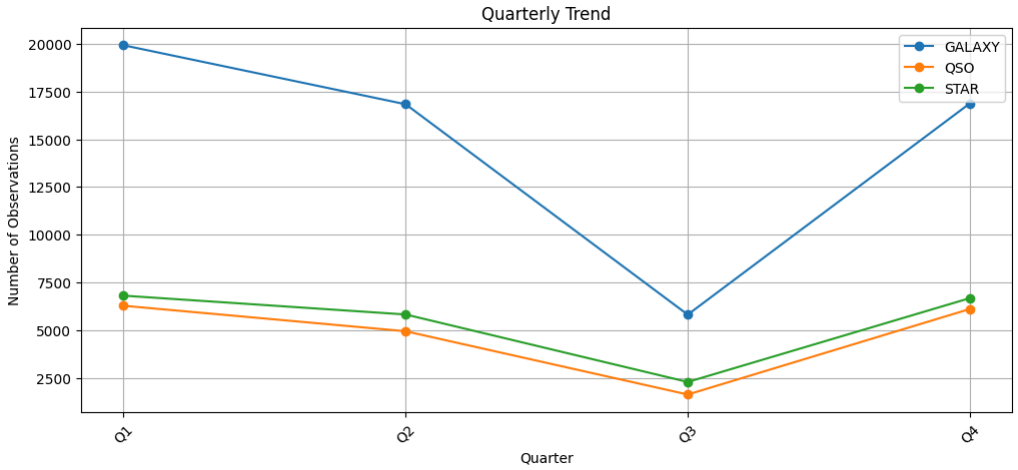
```
plt.legend()

plt.subplot(414)#Residuals
plt.plot(decomposition.resid, label='Residuals', color='black')
plt.legend()

plt.tight_layout()
plt.show()
```

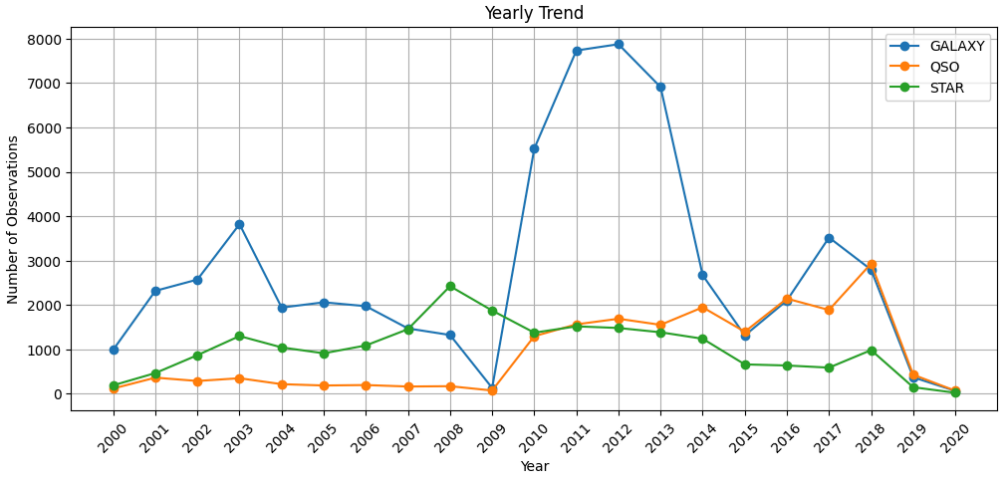


Checking YoY, QoQ, MoM Observations of each class Galaxy, Quasar and Star

```
df['Date'] = pd.to_datetime(df['Date'])

# Extract time features
df['month_name'] = df['Date'].dt.strftime('%B')  # Full month name
df['quarter_name'] = 'Q' + df['Date'].dt.quarter.astype(str)  # 'Q1', 'Q2',
'Q3', 'Q4'
df['year'] = df['Date'].dt.year  # Extract year

# Aggregate by class and month/quarter/year
monthly_counts = df.groupby(['month_name', 'class']).size().unstack(1)
quarterly_counts = df.groupby(['quarter_name', 'class']).size().unstack(1)
yearly_counts = df.groupby(['year', 'class']).size().unstack(1)

# Ensure months and quarters are in the correct order
month_order = ["January", "February", "March", "April", "May", "June",
               "July", "August", "September", "October", "November", "December"]
quarter_order = ["Q1", "Q2", "Q3", "Q4"]

monthly_counts = monthly_counts.reindex(month_order)
quarterly_counts = quarterly_counts.reindex(quarter_order)
yearly_counts.index = yearly_counts.index.astype(int)  # Ensure years remain as
integers

# Function to plot time series
def plot_time_series(data, title, xlabel):
   plt.figure(figsize=(12, 5))

   for col in data.columns:
       plt.plot(data.index, data[col], marker='o', label=col)

   if data.index.dtype == 'int64':  # Yearly plot
       plt.xticks(ticks=data.index, labels=data.index.astype(str), rotation=45)
   else:  # Monthly or Quarterly plot
       plt.xticks(rotation=45)

   plt.xlabel(xlabel)
   plt.ylabel("Number of Observations")
   plt.title(title)
   plt.legend()
   plt.grid()
   plt.show()

plot_time_series(monthly_counts, "Monthly Trend", "Month")
plot_time_series(quarterly_counts, "Quarterly Trend", "Quarter")
plot_time_series(yearly_counts, "Yearly Trend", "Year")
```







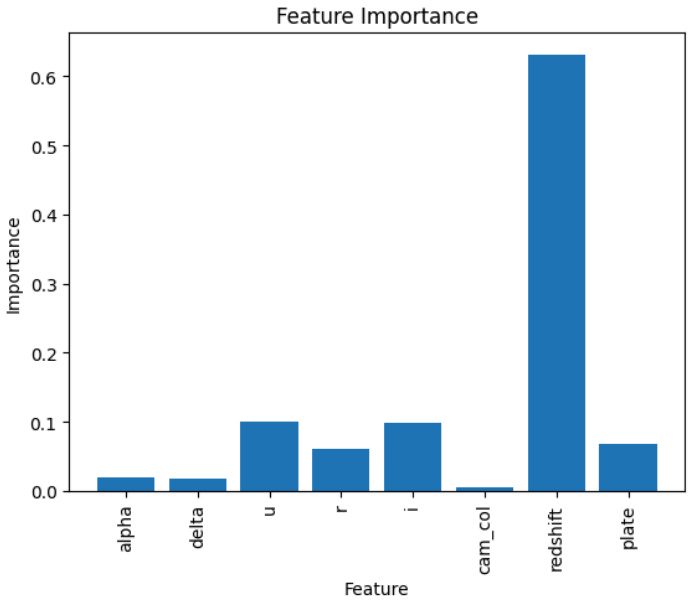
From the MoM and QoQ Analysis we know that the observations are less across June, July and August. Few reasons being: **Weather Conditions** – In many parts of the world, summer months bring more atmospheric turbulence, humidity, and cloudy skies, which reduce visibility. **Shorter Nights** – These months have shorter nights, especially in higher latitudes, limiting the observation window. **Seasonal Scheduling** – Many observatories and astronomers take breaks or conduct maintenance during this period. **Monsoon Season** – In some regions (like India and Southeast Asia), the monsoon season severely affects sky clarity. Through YoY Analysis, we got to know there was a spike in observations 2009 to 2014, but it reduced after that.

Feature Engineering and Feature Selection

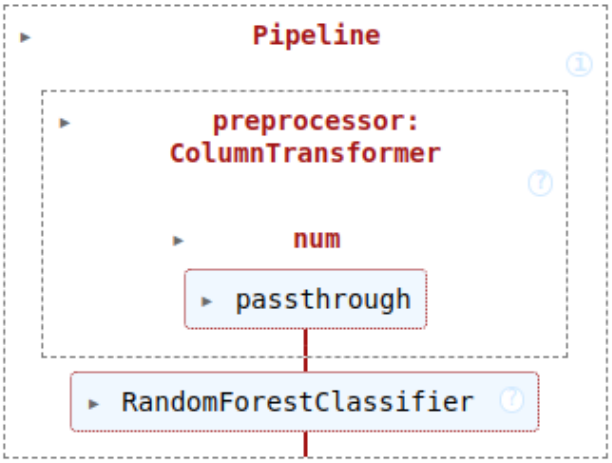
Removing Irrelevant features - Removing ID columns

```
# Removing unnecessary columns
df_ids=[i for i in df.columns if i.__contains__('ID')]
for i in df_ids:
   df=df.drop([i], axis=1)
#Removing columns which are highly correlated
df=df.drop(['g', 'z', 'month_name', 'quarter_name', 'year', 'MJD', 'Date'],
axis=1)
Getting to know feature importance by using Random Forest Classifier
#Checking feature importance using Random Forest Classifier
set_config(display='diagram')
# Separate features and target
X = df.drop('class', axis=1)
y = df['class']
# Label encode the target
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
# Define the preprocessing steps
preprocessor = ColumnTransformer(
   transformers=[
       ('num', 'passthrough', X.columns)  # Pass through all features as they
are
   ])
# Define the model
model = RandomForestClassifier()
# Create the pipeline
pipeline = Pipeline(steps=[
   ('preprocessor', preprocessor),
   ('model', model)
])
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2,
random_state=42)
# Fit the pipeline
pipeline.fit(X_train, y_train)
# Get feature importances
feature_importances = pipeline.named_steps['model'].feature_importances_
pipeline
# Summarize feature importance
feature_names = X.columns
for i, v in enumerate(feature_importances):
   print('Feature: %s, Score: %.5f' % (feature_names[i], v))
# Plot feature importance
plt.bar(range(len(feature_importances)), feature_importances)
plt.xticks(range(len(feature_importances)), feature_names, rotation=90)
plt.xlabel('Feature')
plt.ylabel('Importance')
plt.title('Feature Importance')
plt.show()
Output>>Feature: alpha, Score: 0.01907
Feature: delta, Score: 0.01823
```

```
Feature: u, Score: 0.10023
Feature: r, Score: 0.06009
Feature: i, Score: 0.09775
Feature: cam_col, Score: 0.00491
Feature: redshift, Score: 0.63129
Feature: plate, Score: 0.06844
```



- Among the highly correlated features i and r, r has less feature importance, hence we remove it. Also cam_col has the least feature importance hence we remove it too.



Removing columns with high collinearity and low feature importance

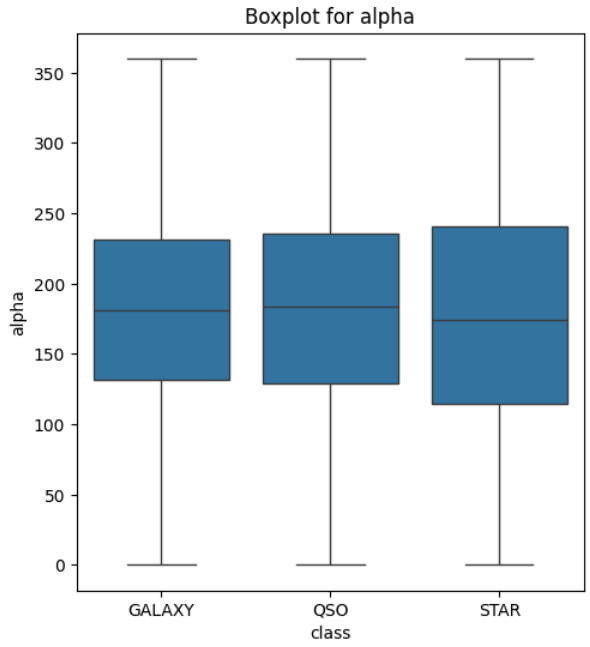
```
df=df.drop(['cam_col', 'r', 'plate'], axis=1)
```

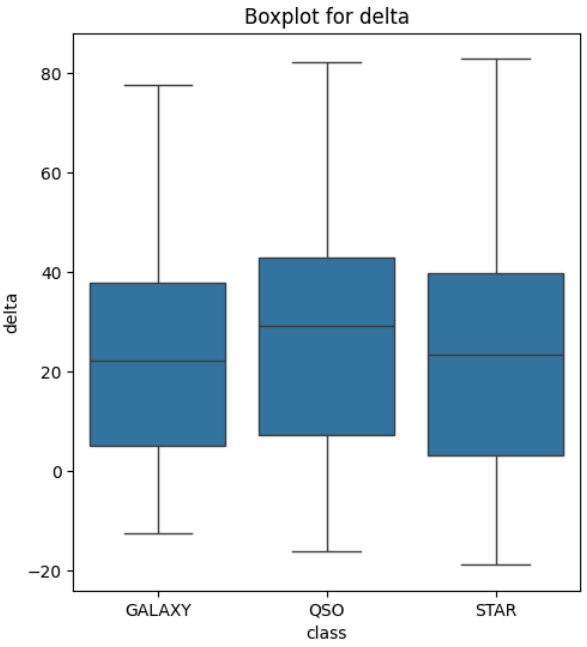
Data Preprocessing

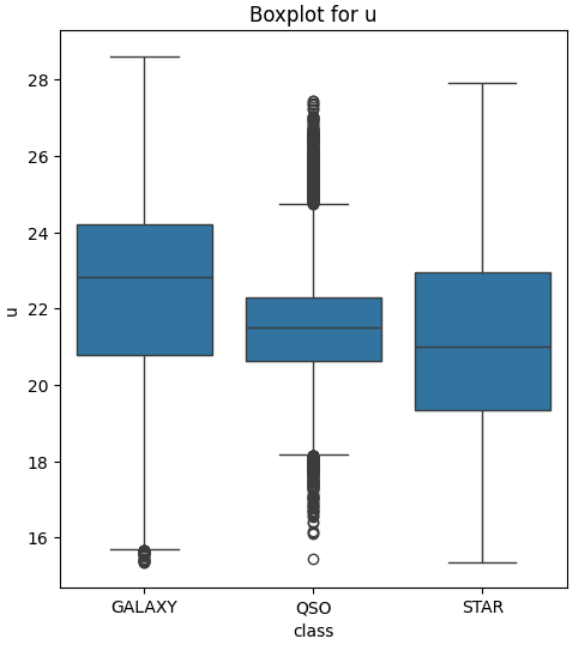
Includes Transformation, Feature Scaling of Numerical Variables and Encoding of Categorical Variables. We saw the presence of outliers in many features, we firstly try to remove them by using mean value imputation.

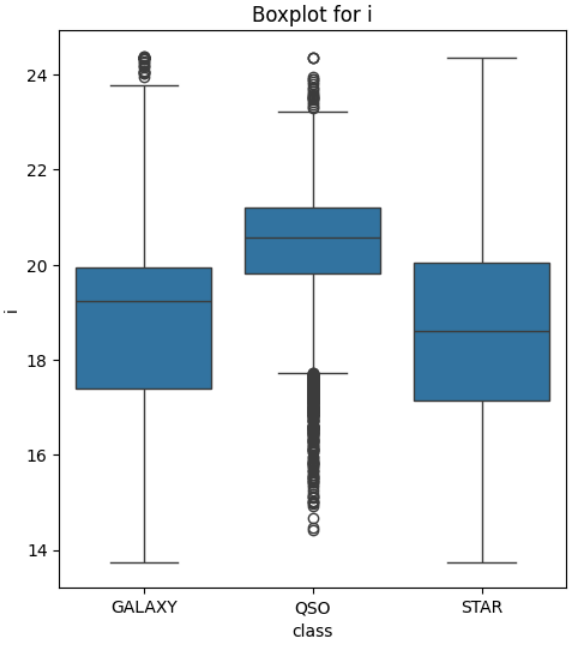
```
X = df.drop(columns='class')
y = df[['class']]
#Handle Outliers by Mean Value imputation
for i in X.columns:
   # Detect outliers using IQR method
   Q1 = X[i].quantile(0.25)
   Q3 = X[i].quantile(0.75)
   IQR = Q3 - Q1
   lower_bound = Q1 - 1.5 * IQR
   upper_bound = Q3 + 1.5 * IQR
   # Identify outliers
   outliers = (X[i] < lower_bound) | (X[i] > upper_bound)
   # Calculate mean without outliers, handling NaN cases
   mean_value = X.loc[~outliers, i].mean()
   mean_value = mean_value if pd.notna(mean_value) else X[i].mean()  #
Fallback
   # Replace outliers with mean
   X.loc[outliers, i] = mean_value
```

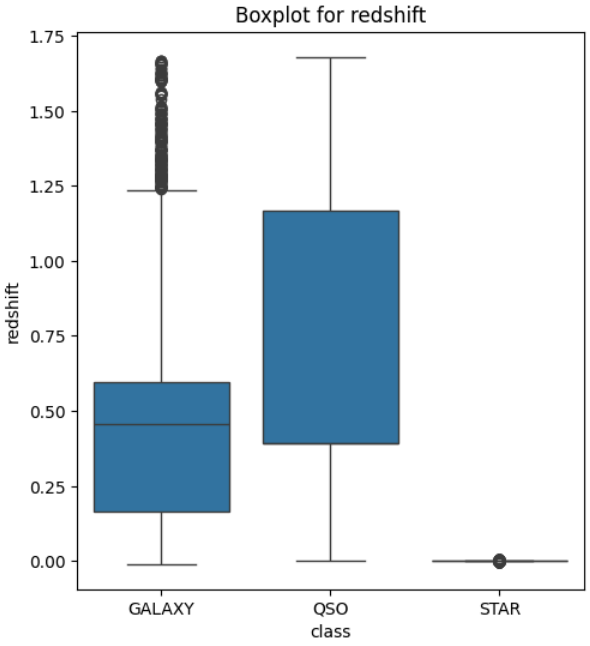
Checking outliers through boxplot after imputation.


Boxplot for alpha
alpha
0
50
100
150
200
250
300
350
GALAXY
QSO
STAR
class


Boxplot for delta
delta
−20
0
20
40
60
80
GALAXY
QSO
STAR
class


Boxplot for u
u
16
18
20
22
24
26
28
GALAXY
QSO
STAR
class


Boxplot for i
i
14
16
18
20
22
24
GALAXY
QSO
STAR
class


Boxplot for redshift
redshift
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
GALAXY
QSO
STAR
class

Even after imputing, outliers were observed particularly in class “Quasar” and “Galaxy”.

Hence, we moved ahead with winsorization for capping the outliers with the Interquartiles, but even after that the outliers didn’t get removed, and we didn’t move ahead with moving any more outliers because in this case we observed that Outliers may not be due to errors in data collection but as the measurements of extremely rare objects like Quasars and Galaxies are naturally extreme.

```
columns_to_winsorize = ['u', 'i', 'redshift']

X_winsorized = X.copy()

# Apply Winsorization
for col in columns_to_winsorize:
   X_winsorized[col] = winsorize(X[col], limits=[0.05, 0.05])
```

After Winsorization the data became more skewed, which would impact the prediction accuracy of the models. Hence the winsorized data was not taken forward.

Yeo-Johnson Transformation was applied for removing skewness in the data. This particular transformation was chosen because it is able to handle negative values present in the dataset, as log, square root and box-cox transformation will fail in these cases.

```
transformer=PowerTransformer('yeo-johnson')#We apply yeo-johnson because it
contains negative values
X_transformed=pd.DataFrame(transformer.fit_transform(X_winsorized),
columns=X.columns)
```

But even after this skewness persisted in the data, by which we can conclude only robust models which can handle both outliers and skewness will give optimum for this dataset.

```
Results for alpha
Shapiro-Wilk Test Statistic:0.9506706276662442
p-value: 9.147652665611731e-94
The data does not follow a normal distribution.
--------------------------------------------------
Results for delta
Shapiro-Wilk Test Statistic:0.9755107001237022
p-value: 4.441525438688982e-77
The data does not follow a normal distribution.
--------------------------------------------------
Results for u
Shapiro-Wilk Test Statistic:0.9667754321558099
p-value: 3.835849913215081e-84
The data does not follow a normal distribution.
--------------------------------------------------
Results for i
Shapiro-Wilk Test Statistic:0.9586322032942416
p-value: 2.052982561377393e-89
The data does not follow a normal distribution.
--------------------------------------------------
Results for redshift
Shapiro-Wilk Test Statistic:0.916755153631144
p-value: 2.5335867981393306e-107
The data does not follow a normal distribution.
```

```
-----------------------------------------------
```

For Encoding, we did not follow the same practise of treating the train and test data separately as, there is no unseen label in both train and test set.

```
label = LabelEncoder()
y = pd.Series(label.fit_transform(y), name="class")
```

Model Selection, Training and Evaluation

Fitting the data from all the preprocessing into the Machine Learning Model.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42) # Splitting into train and test split
scaler=StandardScaler()
X_train_scaled=scaler.fit_transform(X_train)
X_test_scaled=scaler.transform(X_test)
X_train=X_train_scaled
X_test=X_test_scaled
```

For multiclass classification we have chosen to fit the data on the following models viz. Decision Tree, Gaussian Naive Bayes, K Nearest Neighbours, Support Vector Machine, Random Forest, XGBoost, ADABoost and Gradient Boost.

Note regarding SMOTE Analysis and Cross Validation: As the data is highly dimensional i.e. it has 1,00,000 entries performing SMOTE Analysis and Cross Validation using Grid Search CV used to take a lot of time, particularly in the cases of Ensemble Algorithms like Random forest, boosting techniques and Instance based methods like Support Vector Machine.

Hence SMOTE, Cross Validation and Grid Search CV was not included with these algorithms, the drawbacks have been overcome by taking “average” parameters as macro and ensemble algorithms are well suited for handling data imbalance themselves.

When taking the Accuracy, Precision, Recall, F1 Score, Classification Report, ROC-AUC Curve and Precision Recall Curve we must follow either two of strategies: One Vs Rest Strategy (where curves will be separate for each class) or Averaging Strategy. Provided the scale of the database here, the Averaging Strategy approach was chosen to be more suitable.

Support Vector Machine

```
# Define models
svc = SVC(C=1, kernel='linear')
# Fit models on training data
svc.fit(X_train, y_train)
# Predictions
svc_preds = svc.predict(X_test)
# Evaluate models
svc_accuracy = svc.score(X_test, y_test)
# Compute metrics
svc_precision = precision_score(y_test, svc_preds, average="macro")
svc_recall = recall_score(y_test, svc_preds, average="macro")
svc_f1 = f1_score(y_test, svc_preds, average="macro")
# Print results
print(f"SVC Accuracy: {svc_accuracy:.4f}")
print(f"SVC Precision: {svc_precision:.4f}")
```

```
print(f"SVC Recall: {svc_recall:.4f}")
print(f"SVC F1-score: {svc_f1:.4f}")
print("Classification Report for SVC")
print(classification_report(y_test, svc_preds))
```

```
SVC Accuracy: 0.9177
SVC Precision: 0.9021
SVC Recall: 0.9089
SVC F1-score: 0.9054
Classification Report for SVC
              precision    recall  f1-score   support

           0       0.94      0.92      0.93     17845
           1       0.80      0.80      0.80      5700
           2       0.97      1.00      0.99      6455

    accuracy                           0.92     30000
   macro avg       0.90      0.91      0.91     30000
weighted avg       0.92      0.92      0.92     30000
```

```
print("train accuracy", train_accuracy)
print("test_accuracy", test_accuracy)
train accuracy 0.9170
test_accuracy 0.9177
```

With Respect to Support Vector Machine, we observed that the model is performing well, with a linear separator. Accuracy = 91.77%, The model correctly classifies 91.77% of test samples. Precision = 90.21%, On average, 90.21% of predicted positives are correct. Recall = 90.89%, The model correctly identifies 90.89% of actual samples across all classes. F1-score = 90.54%. This suggests the SVC is performing well but may struggle slightly with some classes.

Also the difference between training and testing accuracy is very less. Class 0 (Majority Class) High Precision (0.94) and Recall (0.92) The model handles this class well, though it slightly misses some instances (false negatives). Class 1 (Lowest Performance - 80% Precision & Recall) Lower Precision & Recall (both 0.80). The model struggles with both false positives & false negatives. Class 2 (Best Performance - 97% Precision, 100% Recall) Recall = 1.00 The model identifies all Class 2 samples correctly. Precision = 0.97 → A few false positives, but overall, it performs well. Also the difference between training and testing accuracy is very less.

Random Forest Classifier

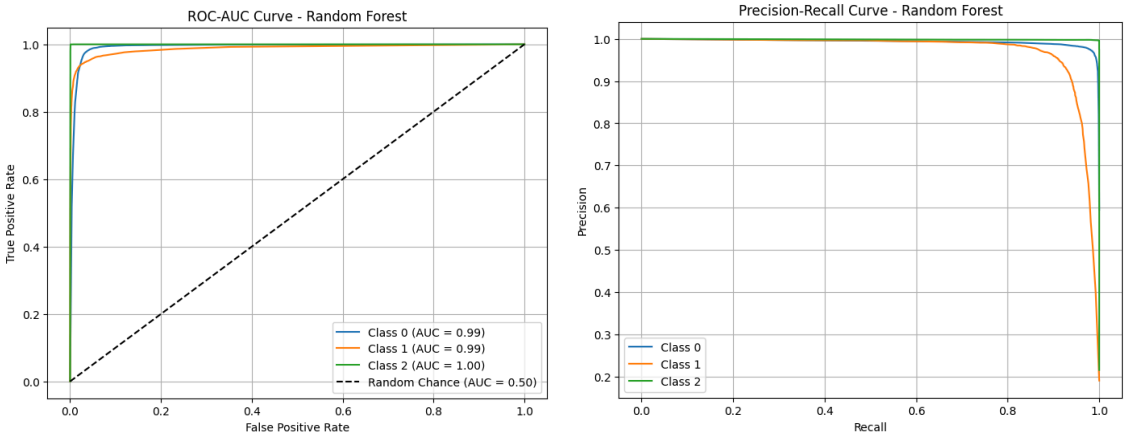
```
rf = RandomForestClassifier(n_estimators=100, max_depth=None,
min_samples_split=2, random_state=42)
rf.fit(X_train, y_train)
rf_preds = rf.predict(X_test)
rf_accuracy = accuracy_score(y_test, rf_preds)
rf_precision = precision_score(y_test, rf_preds, average="macro")
rf_recall = recall_score(y_test, rf_preds, average="macro")
rf_f1 = f1_score(y_test, rf_preds, average="macro")
print(f"Random Forest Accuracy: {rf_accuracy:.4f}")
print(f"Random Forest Precision: {rf_precision:.4f}")
print(f"Random Forest Recall: {rf_recall:.4f}")
print(f"Random Forest F1-score: {rf_f1:.4f}")
```

```
print("Classification Report for Random Forest Classifier")
print(classification_report(y_test, rf_preds))

# Binarize labels for multi-class ROC-AUC & Precision-Recall Curves
y_test_bin = label_binarize(y_test, classes=np.unique(y_test))

# Compute probability scores for ROC & PR curves
rf_proba = rf.predict_proba(X_test)  # Get class probabilities

# Compute ROC-AUC score
roc_auc = roc_auc_score(y_test_bin, rf_proba, average="macro",
multi_class="ovr")
print(f"ROC-AUC Score: {roc_auc:.4f}")

# **Plot ROC Curve**
plt.figure(figsize=(8, 6))
for i in range(len(np.unique(y_test))):
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], rf_proba[:, i])
    plt.plot(fpr, tpr, label=f"Class {i} (AUC = {auc(fpr, tpr):.2f})")

plt.plot([0, 1], [0, 1], "k--", label="Random Chance (AUC = 0.50)")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC-AUC Curve - Random Forest")
plt.legend()
plt.grid()
plt.show()
# **Plot Precision-Recall Curve**
plt.figure(figsize=(8, 6))
for i in range(len(np.unique(y_test))):
    precision, recall, _ = precision_recall_curve(y_test_bin[:, i], rf_proba[:,
i])
    plt.plot(recall, precision, label=f"Class {i}")

plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve - Random Forest")
plt.legend(loc="lower left")
plt.grid()
plt.show()
```

```
Random Forest Accuracy: 0.9734
Random Forest Precision: 0.9733
Random Forest Recall: 0.9648
Random Forest F1-score: 0.9689
Classification Report for Random Forest Classifier
              precision    recall  f1-score   support

           0       0.97      0.98      0.98     17845
           1       0.95      0.91      0.93      5700
           2       1.00      1.00      1.00      6455
```

```
    accuracy                           0.97     30000
   macro avg       0.97      0.96      0.97     30000
weighted avg       0.97      0.97      0.97     30000
```

```
train_accuracy = rf.score(X_train, y_train)
test_accuracy = rf.score(X_test, y_test)
print("train accuracy", train_accuracy)
print("test_accuracy", test_accuracy)

train accuracy 0.9999
test_accuracy 0.9733
```



Accuracy = 97.34%, The model correctly classifies 97.34% of test samples. Precision = 97.28%, On average, 97.28% of predicted positives are correct. Recall = 96.53%, The model correctly identifies 96.53% of actual samples across all classes. F1-score = 96.89%, A balance between precision and recall. Class 0 (Majority Class - 17,845 samples) High precision (0.97) and recall (0.98). The model handles this class very well. Class 1 (Slightly Lower Recall - 91%) Precision = 0.95, Recall = 0.91, Some actual Class 1 samples are misclassified as other classes. Potential Issue: Class 1 might be harder to distinguish or underrepresented in training. Class 2 (Best Performance - 100% Recall & Precision) Perfect precision and recall, The model never misclassified Class 2. Possible Reason: This class might have distinct patterns, making it easier to classify.

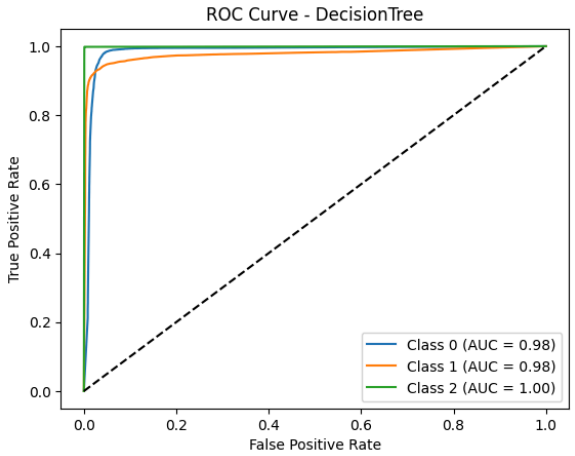
Decision Tree, Gaussian Naive Bayes and K Nearest Neighbours

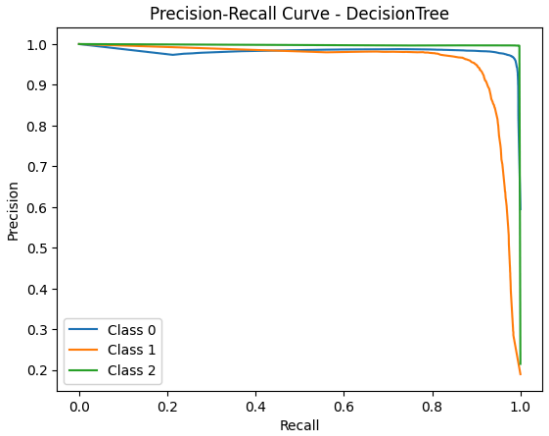
```
# Define models with hyperparameters for tuning
models = {
  "DecisionTree": (Pipeline([
      ("classifier", DecisionTreeClassifier())]),
      {"classifier__max_depth": [5, 10, 20], "classifier__criterion":
["gini", "entropy"]}
  ),
  "Naive_Bayes": (Pipeline([
      ("classifier", GaussianNB())]),
```

```
        {}  # No hyperparameters to tune for GaussianNB
    ),
    "K_Nearest_Neighbours": (Pipeline([
        ("classifier", KNeighborsClassifier())]),
        {"classifier__n_neighbors": [3, 5, 7, 10], "classifier__weights":
["uniform", "distance"]}
    )
}

# Function to train and evaluate models using train_test_split
def evaluate_models(X_train, X_test, y_train, y_test):
    lb = LabelBinarizer()  # For multi-class ROC-AUC & PR curves
    y_test_bin = lb.fit_transform(y_test)  # Convert to binary format
(One-vs-Rest)

    for model_name, (model, params) in models.items():
        print(f"\n🔹 Training {model_name}...\n")

        # Define pipeline and properly reference classifier in hyperparameters
        pipeline = Pipeline([
            ('clf', model)
        ])

        param_grid = {f'clf__{key}': value for key, value in params.items()}

        # Use GridSearchCV to find the best hyperparameters
        grid_search = GridSearchCV(pipeline, param_grid=param_grid,
scoring='accuracy', n_jobs=-1, cv=3)
        grid_search.fit(X_train, y_train)

        # Best model predictions
        best_model = grid_search.best_estimator_
        y_pred = best_model.predict(X_test)
        accuracy = accuracy_score(y_test, y_pred)
        precision = precision_score(y_test, y_pred, average='macro')
        recall = recall_score(y_test, y_pred, average='macro')
        f1 = f1_score(y_test, y_pred, average="macro")  # Fixed f1-score
computation

        # Compute train and test accuracy
        train_accuracy = best_model.score(X_train, y_train)
        test_accuracy = best_model.score(X_test, y_test)

        print(f"Best Params: {grid_search.best_params_}")
        print(f"Train Accuracy: {train_accuracy:.4f}")
        print(f"Test Accuracy: {test_accuracy:.4f}")
        print(f"Accuracy: {accuracy:.4f}")
        print(f"Precision: {precision:.4f}")
        print(f"Recall: {recall:.4f}")
```

```
        print(f"F1-score: {f1:.4f}")
        print(classification_report(y_test, y_pred))

        # ROC-AUC Curve (Only for classifiers with probability output)
        if hasattr(best_model.named_steps['clf'], "predict_proba"):
            y_proba = best_model.predict_proba(X_test)
            roc_auc = roc_auc_score(y_test_bin, y_proba, average="macro",
multi_class="ovr")  # Multi-class ROC-AUC
            print(f"ROC-AUC Score (macro-averaged): {roc_auc:.4f}")

            # Plot ROC Curve
            plt.figure()
            for i in range(len(lb.classes_)):  # Plot each class separately
                fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_proba[:, i])
                plt.plot(fpr, tpr, label=f'Class {lb.classes_[i]} (AUC =
{auc(fpr, tpr):.2f})')

            plt.plot([0, 1], [0, 1], 'k--')  # Diagonal line
            plt.xlabel("False Positive Rate")
            plt.ylabel("True Positive Rate")
            plt.title(f"ROC Curve - {model_name}")
            plt.legend(loc="lower right")
            plt.show()
        else:
            print(f"{model_name} does not support probability predictions,
skipping ROC-AUC.")

        # Precision-Recall Curve (For probability-based classifiers)
        if hasattr(best_model.named_steps['clf'], "predict_proba"):
            plt.figure()
            for i in range(len(lb.classes_)):
                precision, recall, _ = precision_recall_curve(y_test_bin[:,
i], y_proba[:, i])
                plt.plot(recall, precision, label=f'Class {lb.classes_[i]}')

            plt.xlabel("Recall")
            plt.ylabel("Precision")
            plt.title(f"Precision-Recall Curve - {model_name}")
            plt.legend(loc="lower left")
            plt.show()
        else:
            print(f"{model_name} does not support probability predictions,
skipping Precision-Recall Curve.")

# Run evaluation using train_test_split
evaluate_models(X_train, X_test, y_train, y_test)
```

```
Training DecisionTree...

Best Params: {'clf__classifier__criterion': 'entropy',
'clf__classifier__max_depth': 10}
Train Accuracy: 0.9783
Test Accuracy: 0.9705
Accuracy: 0.9705
Precision: 0.9708
Recall: 0.9603
F1-score: 0.9653
              precision    recall  f1-score   support

           0       0.97      0.98      0.98     17845
           1       0.95      0.90      0.92      5700
           2       1.00      1.00      1.00      6455

    accuracy                           0.97     30000
   macro avg       0.97      0.96      0.97     30000
weighted avg       0.97      0.97      0.97     30000

ROC-AUC Score (macro-averaged): 0.9867
```





The Decision Tree classifier achieved a strong accuracy of 97.05%, indicating its ability to correctly classify most instances. With an optimized maximum depth of 10 and the entropy criterion, the model balances complexity and generalization well.

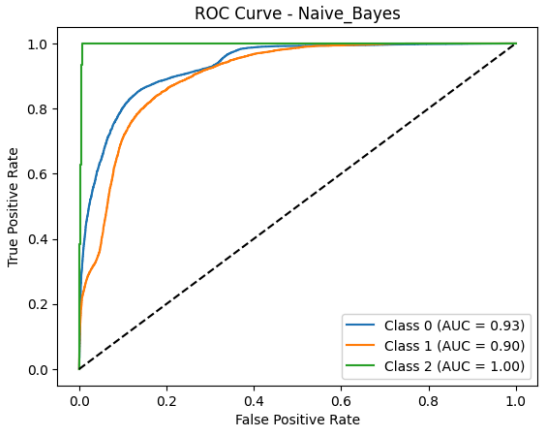
Precision (97.08%): The proportion of correct positive predictions among all predicted positives is high, meaning fewer false positives. Recall (96.05%): The model successfully identifies 96% of actual positive cases, but there may still be some false negatives. F1-Score (90.54%): This combines precision and recall, showing the model maintains a strong balance but might have a slight trade-off due to class imbalances. ROC-AUC Score (98.66%): A very high ROC-AUC suggests excellent separability between classes, meaning the model has strong discriminatory power.

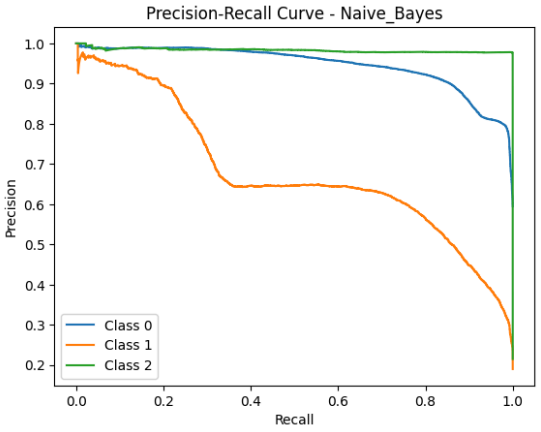
Class 0 (Most Frequent Class): The model performs well here, with high recall and precision. Class 1 (Least Frequent Class): While precision is high (95%), recall drops slightly (90%), meaning some instances of this class are being misclassified. Class 2 (Well-Balanced Class): Perfect scores (100%) indicate the model handles this class very well.

High Accuracy (97.05%): The model is very effective in classification. Excellent Separability (ROC-AUC: 98.66%): The model makes confident predictions with minimal overlap. Strong Performance Across All Classes: Even the minority class (1) is classified with good precision and recall.

Class Imbalance for Class 1: The slightly lower recall (90%) for class 1 suggests that some instances are being misclassified. Overfitting Risk: Decision Trees tend to overfit if too deep. However, with max_depth=10, the model strikes a balance, but this should be checked with cross-validation. Comparing with Other Models: If a model like Random Forest or Gradient Boosting performs better, it might be worth considering ensemble methods.

```
Training Naive_Bayes...

Best Params: {}
Train Accuracy: 0.8538
Test Accuracy: 0.8526
Accuracy: 0.8526
Precision: 0.8294
Recall: 0.8175
F1-score: 0.8225
              precision    recall  f1-score   support

           0       0.86      0.89      0.88     17845
           1       0.65      0.57      0.60      5700
           2       0.98      0.99      0.99      6455

    accuracy                           0.85     30000
   macro avg       0.83      0.82      0.82     30000
weighted avg       0.85      0.85      0.85     30000

ROC-AUC Score (macro-averaged): 0.9439
```





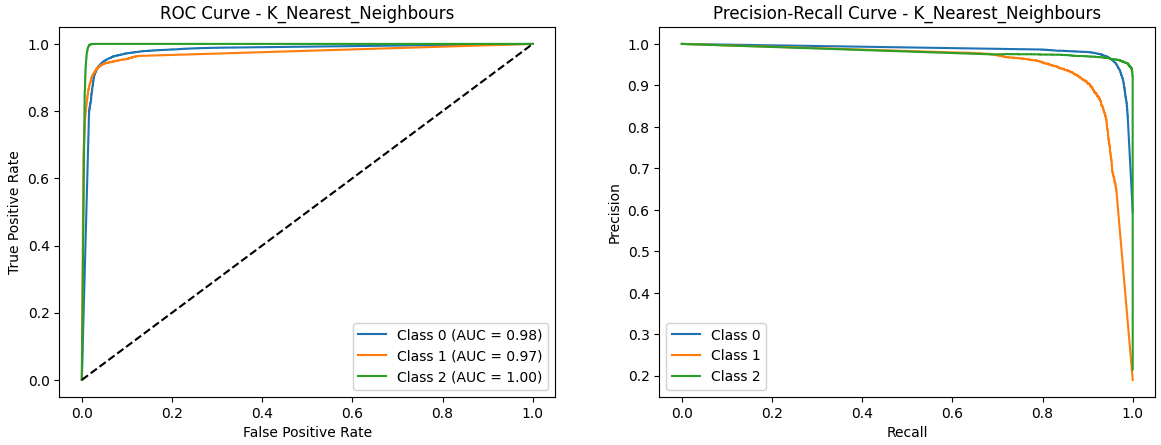
The Naïve Bayes classifier achieved an accuracy of 85.26%, which is lower than more complex models like Random Forest (97.34%) and Decision Tree (97.05%). Given that Naïve Bayes makes strong independence assumptions between features, this result indicates that the dataset likely has feature dependencies that limit Naïve Bayes' effectiveness.

Precision (82.94%): The model has a relatively high precision, meaning it makes a reasonable number of correct positive predictions. Recall (81.75%): The model successfully identifies most of the positive cases, but not as effectively as Decision Trees or Random Forest. F1-Score (82.25%): This score shows a reasonable balance between precision and recall. ROC-AUC Score (94.39%): This suggests that the model still has strong discriminatory power between classes.

Class 0 (Most Frequent Class): High recall (89%) and precision (86%), meaning the model performs well in predicting this class. Class 1 (Least Frequent Class): Significantly lower recall (57%) and precision (65%), indicating the model struggles to classify this minority class correctly. Class 2 (Well-Balanced Class): Near-perfect scores suggest Naïve Bayes is excellent at recognizing this class.

Naive Bayes has all these limitations hence it cannot be considered for prediction.

```
Training K_Nearest_Neighbours...

Best Params: {'clf__classifier__n_neighbors': 10, 'clf__classifier__weights':
'distance'}
Train Accuracy: 1.0000
Test Accuracy: 0.9498
Accuracy: 0.9498
Precision: 0.9414
Recall: 0.9445
F1-score: 0.9426
              precision    recall  f1-score   support

           0       0.96      0.95      0.96     17845
           1       0.92      0.88      0.90      5700
           2       0.94      1.00      0.97      6455

    accuracy                           0.95     30000
   macro avg       0.94      0.94      0.94     30000
weighted avg       0.95      0.95      0.95     30000

ROC-AUC Score (macro-averaged): 0.9824
```



The K-Nearest Neighbors (KNN) classifier achieved an accuracy of 94.98%, The best parameters found were: Number of neighbors 10, Weighting scheme (weights): distance (which gives more weight to closer neighbors)

Precision (94.14%): The model has a high precision, meaning most of its positive predictions are correct. Recall (94.45%): The model successfully identifies most of the actual positive cases. F1-Score (94.26%): The balance between precision and recall is very strong. ROC-AUC Score (98.24%): Indicates that the model has excellent discriminatory power between classes.

Class 0 (Most Frequent Class): Excellent precision (96%) and recall (95%), showing the model performs well in classifying the majority class. Class 1 (Minority Class): Slightly lower recall (88%), meaning some instances of this class are misclassified. Class 2 (Well-Balanced Class): Perfect recall (100%), meaning all class 2 instances were classified correctly.

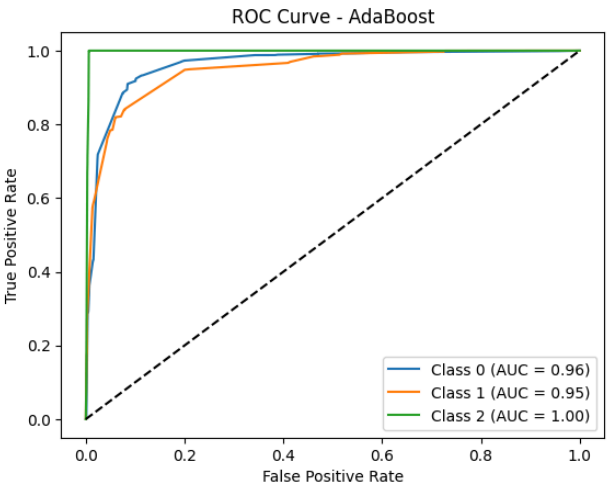
Computational Cost: KNN is expensive in terms of memory and prediction time for large datasets and it its sensitive to Data Scaling: KNN requires proper normalization for best performance.

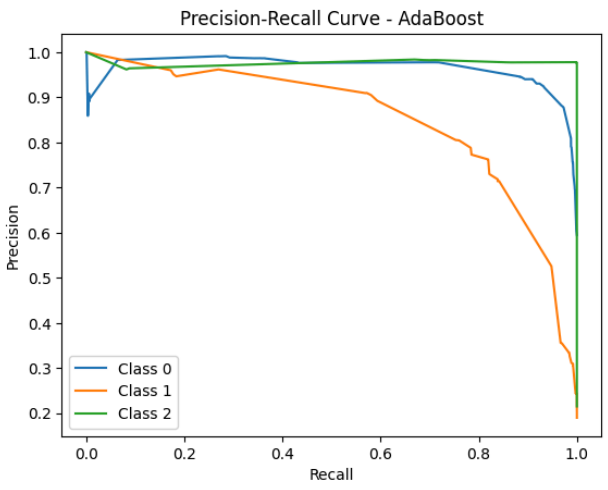
AdaBoost and Gradient Boost

```
models = {
   "AdaBoost": AdaBoostClassifier(n_estimators=100, learning_rate=0.1),
   "Gradient Boosting": GradientBoostingClassifier(n_estimators=100,
learning_rate=0.1, max_depth=5)}
# Function to train and evaluate models
def evaluate_models(X_train, X_test, y_train, y_test):
   lb = LabelBinarizer()
   y_test_bin = lb.fit_transform(y_test)
   for model_name, model in models.items():
       print(f"\n🔹 Training {model_name}...\n")
       # Define pipeline
       pipeline = Pipeline([
           ('clf', model)])
       # Train the model
       pipeline.fit(X_train, y_train)
       y_pred = pipeline.predict(X_test)
       accuracy = accuracy_score(y_test, y_pred)
       precision = precision_score(y_test, y_pred, average='macro')
       recall = recall_score(y_test, y_pred, average='macro')
       f1 = f1_score(y_test, y_pred, average="macro")
       print(f"Accuracy: {accuracy:.4f}")
       print(f"Precision: {precision:.4f}")
       print(f"Recall: {recall:.4f}")
       print(f"F1 score: {f1:.4f}")
       print(classification_report(y_test, y_pred))
       # ROC-AUC Curve (Only for classifiers with probability output)
       if hasattr(pipeline.named_steps['clf'], "predict_proba"):
           y_proba = pipeline.predict_proba(X_test)
           roc_auc = roc_auc_score(y_test_bin, y_proba, average="micro",
multi_class="ovr")  # Multi-class ROC-AUC
           print(f"ROC-AUC Score (micro-averaged): {roc_auc:.4f}")
           # Plot ROC Curve
           plt.figure()
           for i in range(len(lb.classes_)):
               fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_proba[:, i])
               plt.plot(fpr, tpr, label=f'Class {lb.classes_[i]} (AUC =
{auc(fpr, tpr):.2f})')
           plt.plot([0, 1], [0, 1], 'k--')  # Diagonal line
           plt.xlabel("False Positive Rate")
           plt.ylabel("True Positive Rate")
           plt.title(f"ROC Curve - {model_name}")
           plt.legend(loc="lower right")
           plt.show()
       else:
           print(f"{model_name} does not support probability predictions,
skipping ROC-AUC.")
       # Precision-Recall Curve (For probability-based classifiers)
       if hasattr(pipeline.named_steps['clf'], "predict_proba"):
           plt.figure()
```

```
            for i in range(len(lb.classes_)):
                precision, recall, _ = precision_recall_curve(y_test_bin[:,
i], y_proba[:, i])
                plt.plot(recall, precision, label=f'Class {lb.classes_[i]}')
            plt.xlabel("Recall")
            plt.ylabel("Precision")
            plt.title(f"Precision-Recall Curve - {model_name}")
            plt.legend(loc="lower left")
            plt.show()
        else:
            print(f"{model_name} does not support probability predictions,
skipping Precision-Recall Curve.")
evaluate_models(X_train, X_test, y_train, y_test)
```

```
Training AdaBoost...
Train Accuracy: 0.8382
Test Accuracy: 0.8362
Accuracy: 0.8362
Precision: 0.9085
Recall: 0.7205
F1 score: 0.7194
              precision    recall  f1-score   support

           0       0.79      0.99      0.88     17845
           1       0.96      0.17      0.29      5700
           2       0.98      1.00      0.99      6455

    accuracy                           0.84     30000
   macro avg       0.91      0.72      0.72     30000
weighted avg       0.86      0.84      0.79     30000
ROC-AUC Score (micro-averaged): 0.9709
```



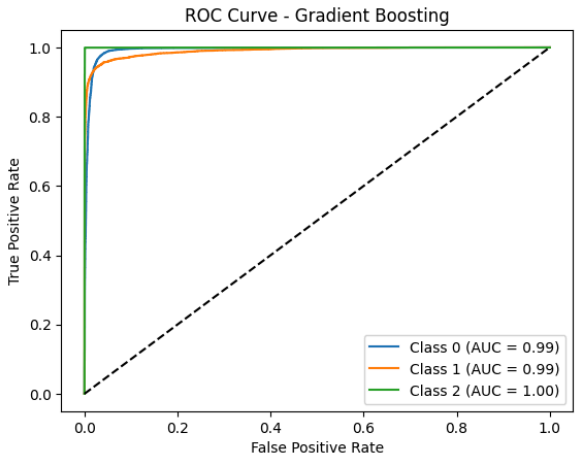


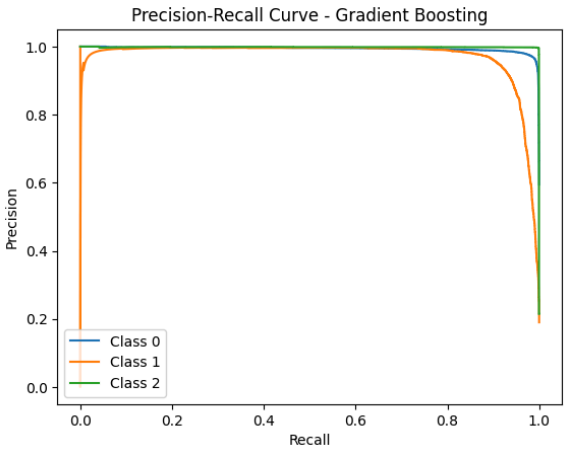
The AdaBoost classifier achieved an accuracy of 83.62%. Though it has a high precision (90.85%), its recall (72.05%) is relatively low, indicating that while AdaBoost is good at making correct positive predictions, it struggles to detect all positive cases.

Class 0 (Most Frequent Class): Very high recall (99%), meaning it correctly classifies almost all instances of this class. Class 1 (Minority Class): Very low recall (17%), meaning AdaBoost struggles to correctly classify many class 1 instances. Class 2 (Well-Balanced Class): Perfect recall (100%), meaning all class 2 instances were classified correctly.

The AdaBoost classifier performs well in distinguishing classes in general, but fails significantly on the minority class (Class 1). While it has a strong ROC-AUC score (97.09%), its low recall (17%) for Class 1 means it should not be relied upon for applications where detecting minority cases is critical.

```
Training Gradient Boosting...

Train Accuracy: 0.9784
Test Accuracy: 0.9729
Accuracy: 0.9729
Precision: 0.9726
Recall: 0.9643
F1 score: 0.9683
              precision    recall  f1-score   support

           0       0.97      0.98      0.98     17845
           1       0.95      0.91      0.93      5700
           2       1.00      1.00      1.00      6455

    accuracy                           0.97     30000
   macro avg       0.97      0.96      0.97     30000
weighted avg       0.97      0.97      0.97     30000

ROC-AUC Score (micro-averaged): 0.9969
```





The Gradient Boosting classifier has achieved an accuracy of 97.30%, which is the highest among the models tested so far. High Precision (97.26%): The model makes very few false positive errors. High Recall (96.44%): It correctly identifies most positive cases, making it highly reliable. Outstanding ROC-AUC Score (99.69%): Indicates that the classifier ranks positive instances exceptionally well.

Class 0 (Majority Class): Strong performance (98% recall, 98% F1-score), meaning the model correctly classifies most of these instances. Class 1 (Minority Class): High recall (91%)—much better than AdaBoost's 17% recall for the same class, meaning Gradient Boosting significantly improves minority class classification. Class 2 (Well-Balanced Class): Perfect recall (100%), showing that model never misses instances from class 2.

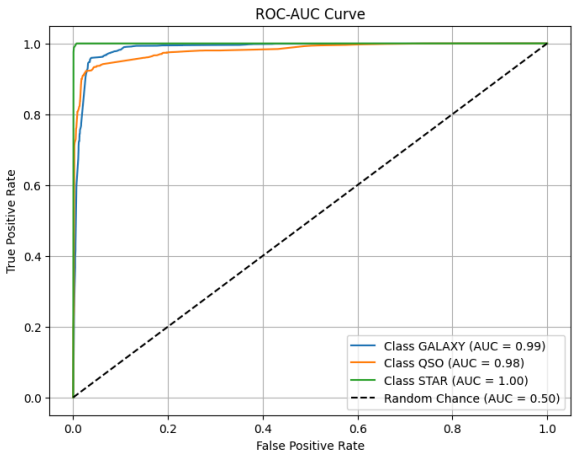
Gradient Boosting is the best-performing model in this evaluation, achieving the highest accuracy (97.30%) and ROC-AUC score (99.69%), while also effectively handling minority class instances. If computational resources allow, this model is an excellent choice for deployment.

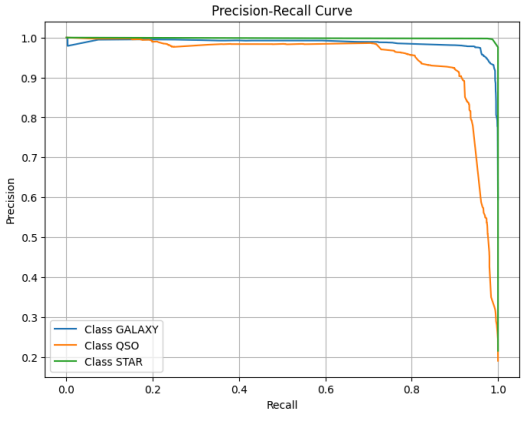
XG Boost

```
# Initialize XGBoost classifier
model = xgb.XGBClassifier(objective='multi:softmax',
num_class=len(label_encoder.classes_),
                          n_estimators=100, learning_rate=0.01, max_depth=3,
random_state=42)

# Train the model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Decode predictions back to original labels (if needed)
y_pred_labels = label_encoder.inverse_transform(y_pred)

# Evaluate metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro')
recall = recall_score(y_test, y_pred, average='macro')
f1 = f1_score(y_test, y_pred, average='macro')

# Print metrics
print(f"Accuracy: {accuracy :.4f}%")
print(f"Precision: {precision :.4f}%")
print(f"Recall: {recall :.4f}%")
print(f"F1-Score: {f1 :.4f}%")

print("Classification Report:")
print(classification_report(y_test, y_pred,
target_names=label_encoder.classes_))

# Binarize labels for multi-class ROC-AUC & Precision-Recall Curves
y_test_bin = label_binarize(y_test,
classes=range(len(label_encoder.classes_)))
# Compute probability scores for ROC & PR curves
y_proba = model.predict_proba(X_test)  # Get class probabilities

# Compute ROC-AUC score
roc_auc = roc_auc_score(y_test_bin, y_proba, average="macro",
multi_class="ovr")
print(f"ROC-AUC Score: {roc_auc:.4f}")
# Plot ROC Curve
plt.figure(figsize=(8, 6))
for i in range(len(label_encoder.classes_)):
```

```
    fpr, tpr, _ = roc_curve(y_test_bin[:, i], y_proba[:, i])
    plt.plot(fpr, tpr, label=f"Class {label_encoder.classes_[i]} (AUC =
{auc(fpr, tpr):.2f})")
plt.plot([0, 1], [0, 1], "k--", label="Random Chance (AUC = 0.50)")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC-AUC Curve")
plt.legend()
plt.grid()
plt.show()
# Precision-Recall Curve
plt.figure(figsize=(8, 6))
for i in range(len(label_encoder.classes_)):
    precision, recall, _ = precision_recall_curve(y_test_bin[:, i],
y_proba[:, i])
    plt.plot(recall, precision, label=f"Class {label_encoder.classes_[i]}")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
plt.legend(loc="lower left")
plt.grid()
plt.show()
```

```
Training AdaBoost...

Train Accuracy: 0.8382
Test Accuracy: 0.8362
Accuracy: 0.8362
Precision: 0.9085
Recall: 0.7205
F1 score: 0.7194
              precision    recall  f1-score   support

           0       0.79      0.99      0.88     17845
           1       0.96      0.17      0.29      5700
           2       0.98      1.00      0.99      6455

    accuracy                           0.84     30000
   macro avg       0.91      0.72      0.72     30000
weighted avg       0.86      0.84      0.79     30000

ROC-AUC Score (micro-averaged): 0.9709
```





The XGBoost classifier has achieved an accuracy of 94.71%, making it a highly effective model for the given classification task. High Precision (95.65%): The model makes very few false positive errors. Strong Recall (92.09%): The classifier successfully identifies most positive instances. Excellent ROC-AUC Score (98.88%): Indicates a strong ability to distinguish between different classes.

GALAXY (Majority Class): High recall (98%) ensures that most galaxies are correctly identified. QSO (Minority Class): Precision (96%) is high, but recall (78%) is relatively lower—indicating that some quasars are misclassified. STAR (Well-Represented Class): Near-perfect recall (100%) ensures that virtually no stars are misclassified. XGBoost is a strong performer, achieving 94.71% accuracy and a 98.88% ROC-AUC score, making it one of the best models for this classification problem. While its recall for QSO is slightly lower, further tuning could enhance performance and reduce misclassification.

Table 1. *Summary of Results*

	Train Accuracy	**Test Accuracy**	**Precision**	**Recall**	**F1 Score**
SVM Classifier	0.9170	0.9177	0.9021	0.9089	0.9054
RF Classifier*	0.9999	0.9733	0.9728	0.9650	0.9688
DT Classifier	0.9783	0.9705	0.9707	0.9603	0.9653
GNB Classifier	0.8538	0.8526	0.8294	0.8175	0.8225
KNN Classifier	1.0000	0.9498	0.9414	0.9445	0.9426
AdaBoost Classifier	0.8382	0.8362	0.9085	0.7205	0.7194
GB Classifier*	0.9784	0.9729	0.9726	0.9644	0.9684
XGBoost Classifier	0.8382	0.8362	0.9565	0.9209	0.9354

Best Overall Performers:

- **Random Forest (RF) and Gradient Boosting (GB)** have the highest overall metrics, with RF slightly leading. RF: **Accuracy (0.9733), F1 Score (0.9688)** GB: **Accuracy (0.9730), F1 Score (0.9684)**
- **Decision Tree (DT) Performance**: DT has strong scores but slightly lower than RF and GBM, which is expected since RF is an ensemble of DTs.

Weakest Performer:

- **Gaussian Naïve Bayes (GNB)** has the lowest overall performance, especially Recall (0.8175) and F1 Score (0.8225), likely due to its assumption of feature independence.

AdaBoost's Precision vs. Recall:

- AdaBoost has **very high precision (0.9085)** but **low recall (0.7205)**, meaning it is more conservative in making positive predictions but misses some actual positives.

SVM vs. KNN:

- KNN outperforms SVM in **all metrics**, suggesting that in this dataset, distance-based classification is more effective than hyperplane-based separation.

XGBoost vs. RF/GBM:
- XGBoost is strong but has slightly lower Recall (0.9209) than RF and GBM, meaning it may be more prone to false negatives.

Key Metric:

In our project, both false positives and false negetives can be costly, reasons being:

A false positive and false negative in stellar classification means that an object is misclassified as a type that it is not. The cost of this depends on the application:

- **Astronomical Surveys**: If a star is misclassified as a rare type (e.g., a white dwarf misclassified as a neutron star), it could lead to wasted observational resources.
- **Exoplanet Searches**: If a star is wrongly classified as a Sun-like star when it is actually a giant, it could mislead exoplanet studies.
- **Astrophysical Modeling**: Misclassifications can introduce biases in large-scale models of stellar evolution.

MODEL DEPLOYMENT

For creating a User Interface for the end user we made use of Streamlit, where following features were integrated into the app:

1. Explore the Dataset by viewing the raw dataset, feature description, feature profile
2. Select a classification model among various models, tuning various hyper parameters of the model, view their results such as accuracy, precision, recall, ROC Curve and Precision-Recall Curve

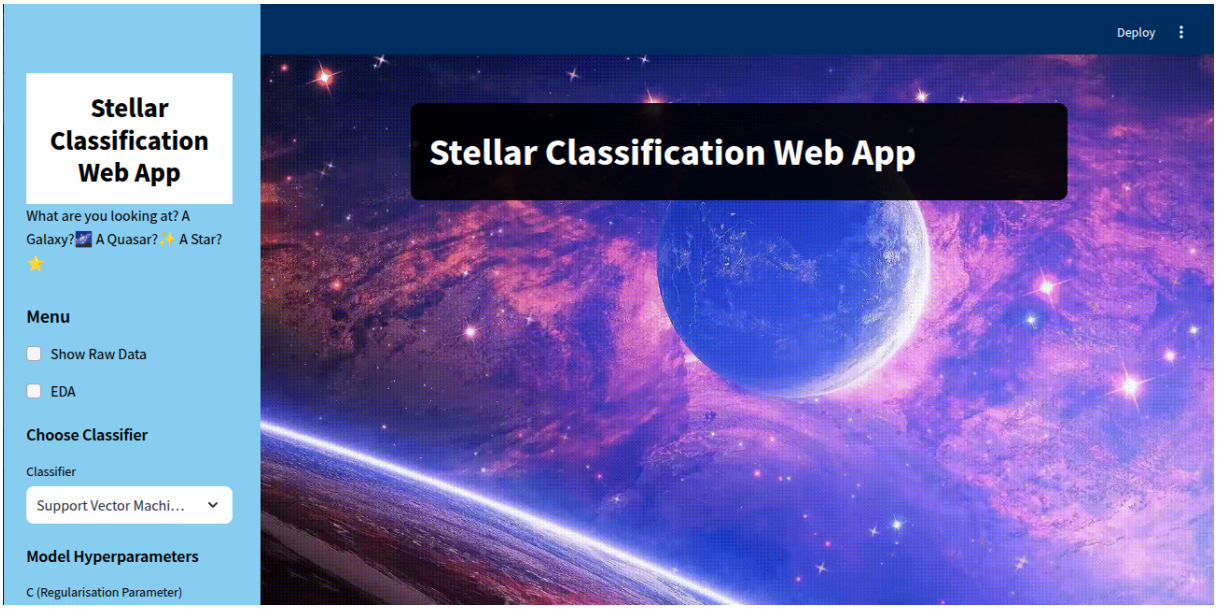


Image 2: Landing Page of the Application

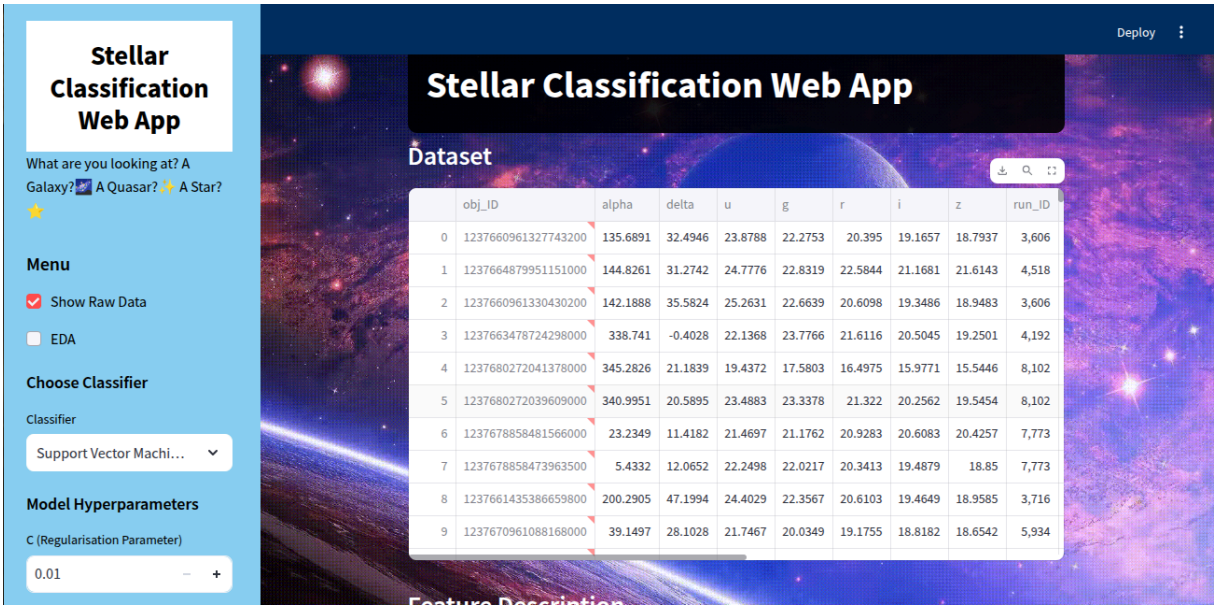


Image 3: Viewing the Raw Data

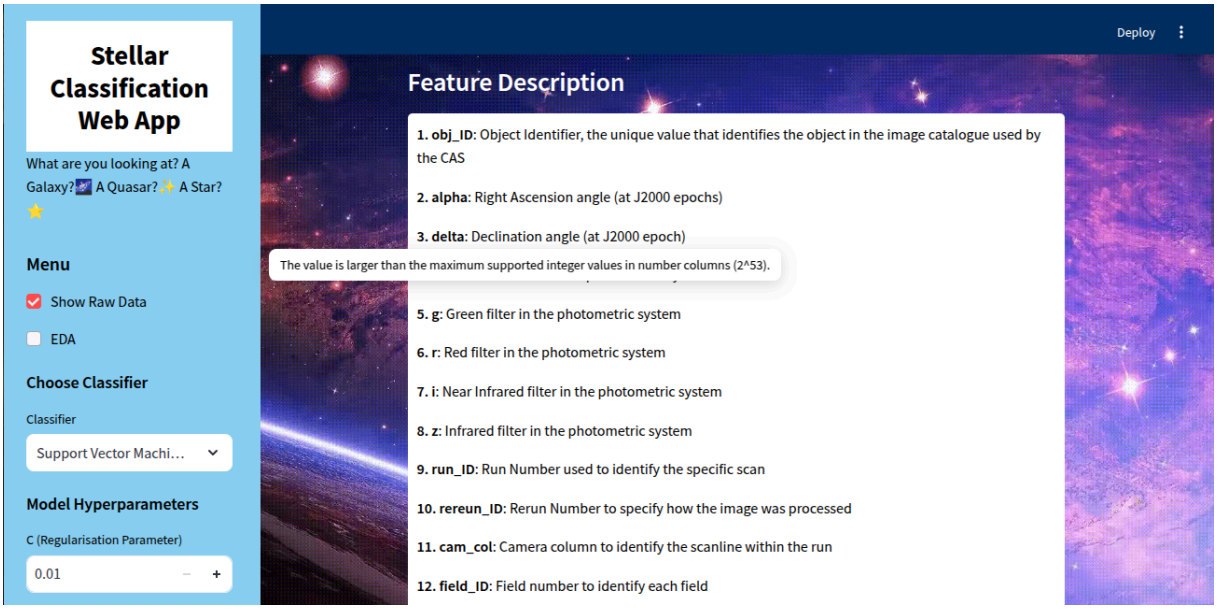


Image 4: Viewing Feature Description

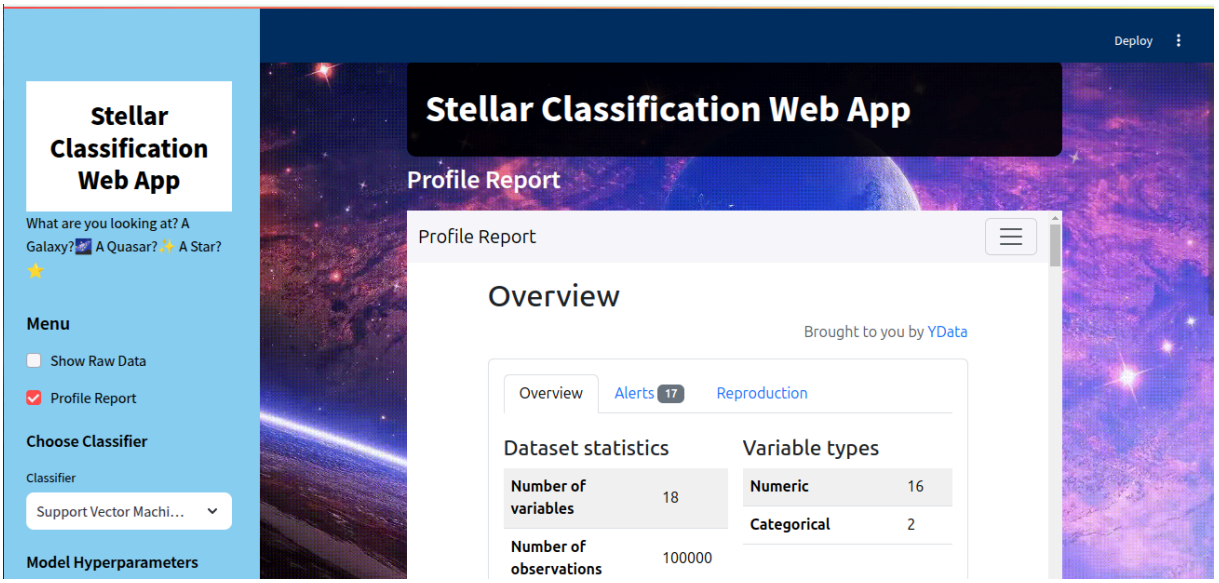


Image 5: Viewing Profile Report

Image 6: Selecting ML Model and tuning hyper parameters manually

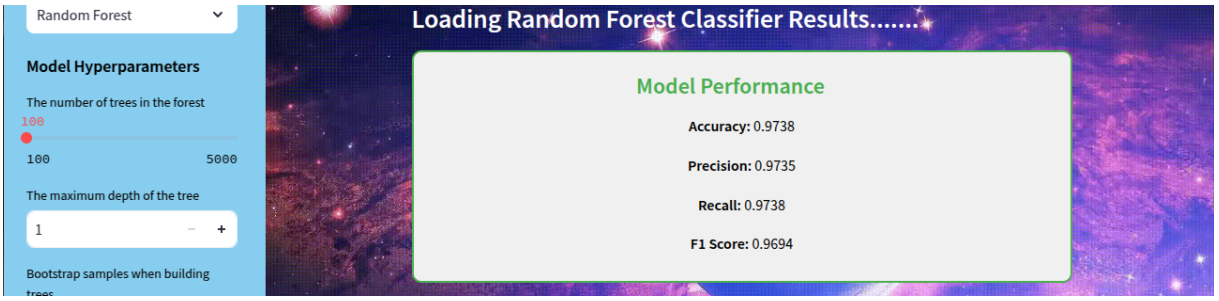


Image 5: Viewing Accuracy, Precision, Recall and F1-Score for the fitted Model

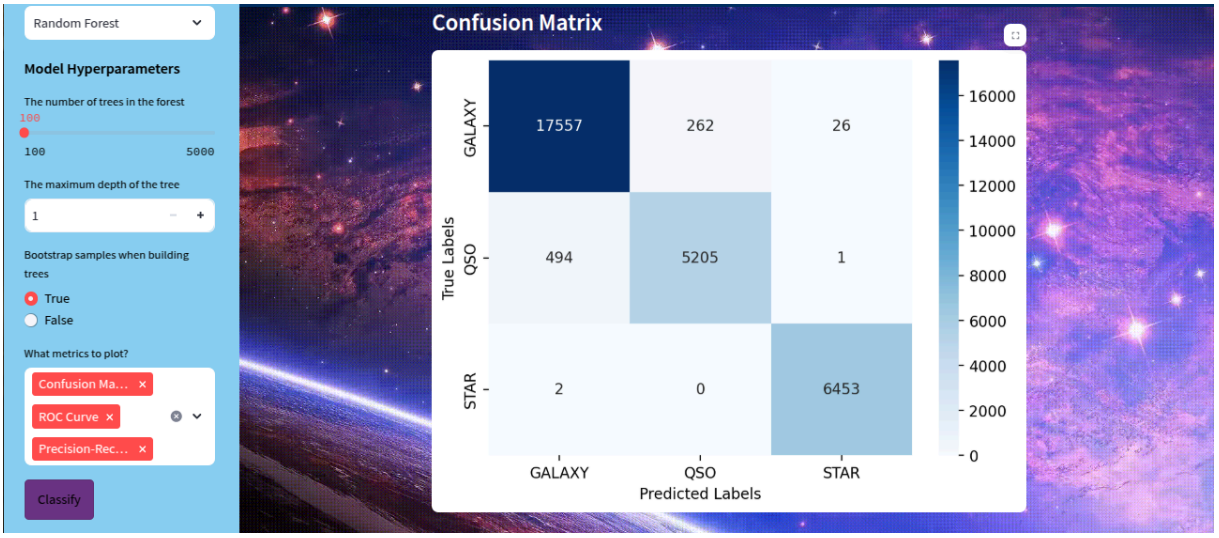


Image 7: Viewing the Confusion Matrix of the fitted Model

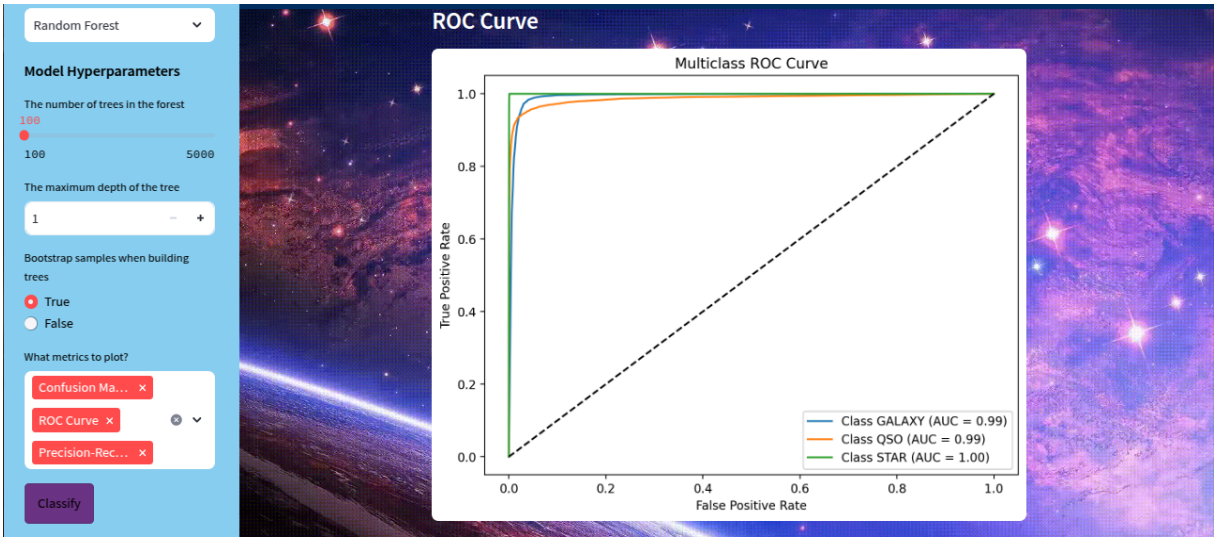


Image 8: Viewing the ROC Curve for the fitted Model

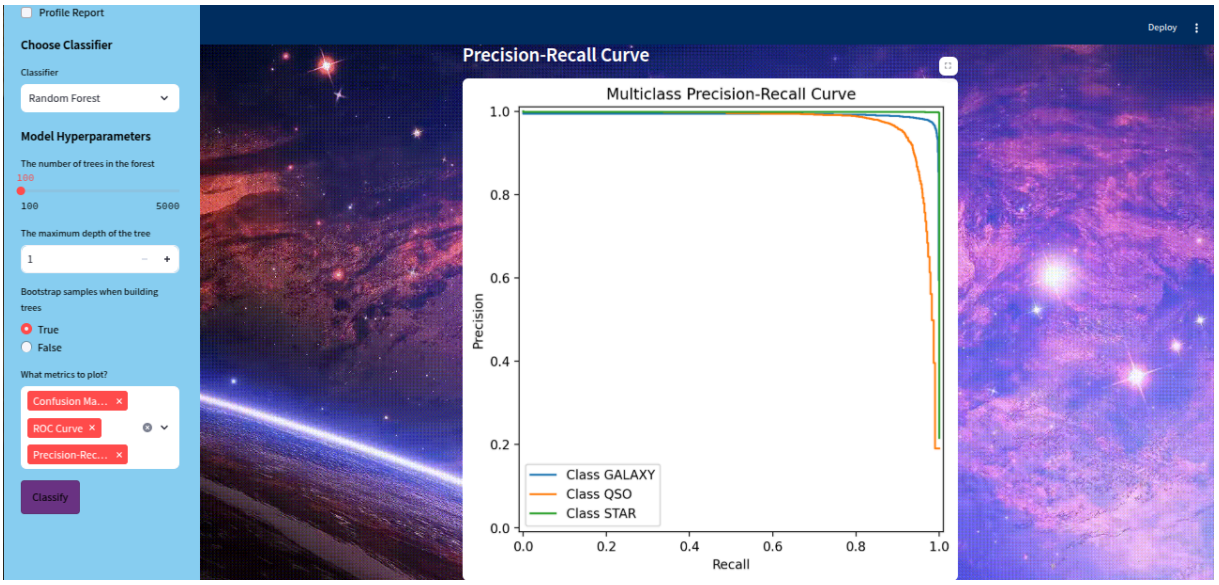


Image 9: Viewing the Precision Recall Curve of the fitted Model

CONCLUSIONS

By implementing the Machine Learning Algorithms for the Dataset, we got to know

1. Overall all models perform well, with a good accuracy and precision, whereas it was observed that recall was less across models, RF Classifier and GB Boost was able to classify positive examples correctly in comparison with other models, i.e. slightly fewer false negatives.
2. When we look at the class wise performance across models, the models were able to correctly classify Stars (perfectly in some cases) where it means that they had values which were easily distinguishable. The models were also predicting Galaxies satisfactorily, but Quasars were not easily identifiable by the models, some reasons them being very rare objects, more outliers observed and comparatively less representation in the data (Minority Class)
3. When we look into different evaluation metrics F1 Score, Precision and Recall are of Primary importance. Precision helps us to minimize false positives (e.g., avoid misclassifying a Quasar as a Star). Recall ensures all objects of a certain class are correctly detected (e.g., ensuring all quasars are classified correctly) and F1-score shows a good balance between precision and recall, which is specially useful in this case as distribution is skewed.
4. Precision for minority classes is primarily important in this case, because the cost of wrongly classifying an object is costly (in this case of Quasars).
5. GB Boost is the best model for stellar classification particularly considering precision for minority classes.

ANNEXURE - A

ABBREVIATIONS

1. SVM - Support Vector Machine
2. RF - Random Forest
3. DT - Decision Tree
4. GNB - Gaussian Naive Bayes
5. KNN - K Nearest Neigbours
6. ROC AUC Curve - Receiver Operating Curve Area Under the Curve
7. SDSS - Sloan Digital Sky Survey
8. CAS - Centre for Astrophysics and Supercomputing
9. MJD - Mean Julian Date
10. UGRIZ - Ultraviolet, Green, Red, Near Infrared, Infrared

REFERENCES

1. https://astro.unl.edu/naap/hr/hr_background1.html#:~:text=Astronomers%20have%20devised%20a%20classification,Girl%2FGuy%2C%20Kiss%20Me!
2. https://academic.oup.com/mnras/article/520/2/2269/7000842
3. https://www.celestron.com/blogs/knowledgebase/what-are-ra-and-dec#:~:text=RA%20(right%20ascension)%20and%20Dec,like%20latitude%20on%20the%20Earth.
4. https://www.envinsci.co.uk/what-is-photometry-and-how-are-optical-filters-involved/#:~:text=A%20photometric%20system%20defines%20a,are%20comparable%20across%20different%20observations.
5. https://skyserver.sdss.org/dr1/en/proj/advanced/color/sdssfilters.asp
6. https://machinelearningmastery.com/calculate-feature-importance-with-python/
7. www.celestron.com
8. geeks for geeks
9. w3schools